



P4₁₆ Portable Switch Architecture (PSA)

Version v1.2, 2026-09-17

Contents

1. Target Architecture Model	3
2. Naming conventions	4
3. Packet paths	4
4. PSA Data types	6
4.1. PSA type definitions	6
4.2. PSA supported metadata types	8
4.3. Match kinds	9
4.3.1. Range tables	9
4.3.2. Ternary tables	10
4.3.3. Longest prefix match tables	10
4.3.4. Exact match tables	10
5. Data plane vs. control plane data representations	11
6. Programmable blocks	14
7. Packet Path Details	15
7.1. Initial values of packets processed by ingress	15
7.2. Initial packet contents for packets from ports	15
7.2.1. Initial packet contents for resubmitted packets	16
7.2.2. Initial packet contents for recirculated packets	16
7.2.3. User-defined metadata for all ingress packets	16
7.3. Behavior of packets after ingress processing is complete	17
7.3.1. Multicast replication	19
7.4. Actions for directing packets during ingress	19
7.4.1. Unicast operation	20
7.4.2. Multicast operation	20
7.4.3. Drop operation	21
7.5. Initial values of packets processed by egress	21
7.5.1. Initial packet contents for normal packets	22
7.5.2. Initial packet contents for packets cloned from ingress to egress	22
7.5.3. Initial packet contents for packets cloned from egress to egress	22
7.5.4. User-defined metadata for all egress packets	22
7.5.5. Multicast and clone copies	23
7.6. Behavior of packets after egress processing is complete	23
7.7. Actions for directing packets during egress	24
7.7.1. Drop operation	24
7.8. Contents of packets sent out to ports	25
7.9. Packet Cloning	25
7.9.1. Clone Examples	26
7.10. Packet Resubmission	27
7.11. Packet Recirculation	28
8. PSA Externs	29
8.1. Restrictions on where externs may be used	29
8.2. PSA Table Properties	30
8.2.1. Table entry timeout notification	31
8.3. Packet Replication Engine	32
8.4. Buffering Queuing Engine	33
8.5. Hashes	33
8.5.1. Hash function	33
8.6. Checksums	34

8.6.1. Basic checksum	34
8.6.2. Incremental checksum	34
8.6.3. InternetChecksum examples	35
8.7. Counters	39
8.7.1. Counter types	40
8.7.2. Counter	40
8.7.3. Direct Counter	40
8.7.4. Example program using counters	41
8.8. Meters	42
8.8.1. Meter types	44
8.8.2. Meter colors	44
8.8.3. Meter	44
8.8.4. Direct Meter	44
8.9. Registers	44
8.10. Random	47
8.11. Action Profile	47
8.11.1. Action Profile Example	48
8.12. Action Selector	49
8.12.1. Group Actions	49
8.12.2. Action Selector Example	51
8.13. Timestamps	51
8.14. Packet Digest	53
9. Atomicity of control plane API operations	57
Appendix A: Open Issues	58
A.1. Action Selectors	58
A.2. Observation and control of congestion	58
A.3. Enabling full implementation of In-band Network Telemetry	58
A.4. PSA profiles	58
Appendix B: Implementation of the InternetChecksum extern	60
Appendix C: Example implementation of Counter extern	61
Appendix D: Rationale for design	62
D.1. Why egress processing?	62
D.2. No output port change during egress	63
D.3. Ingress deparser and egress parser	64
Appendix E: Multi-pipeline PSA devices	64
Appendix F: Packet ordering	67
Appendix G: Supporting empty action selector groups	69
Appendix H: PSA Reserved Annotations	71
Appendix I: Revision History	72
I.1. Changes made in version 1.2	72
I.1.1. Add match_kind optional	72
I.1.2. Remove control plane API function signatures	72
I.2. Changes made in version 1.1	72
I.2.1. Numeric translation between P4Runtime API values and data plane values	73
I.2.2. Add the ability for packet clone sessions to create multiple copies	73
I.2.3. Add psa_idle_timeout table property	73
I.2.4. Add psa_empty_group_action table property	73
I.2.5. Other changes	74
I.2.6. Changes to the psa.p4 include file	74
I.2.7. Changes to example PSA programs in the p4-16/psa/examples directory	74

Abstract

P4 is a language for expressing how packets are processed by the data plane of a programmable network forwarding element. P4 programs specify how the various programmable blocks of a target architecture are programmed and connected. The Portable Switch Architecture (PSA) is a target architecture that describes common capabilities of network switch devices that process and forward packets across multiple interface ports.

1. Target Architecture Model

As an analogy, the PSA is to the P4₁₆ language as the C standard library is to the C programming language. PSA defines a library of types, P4₁₆ externs for frequently used constructs such as counters, meters, and registers, and a set of "packet paths" that enable you to write P4 programs that control the flow of packets in a packet switch that has multiple ports, e.g. dozens of Ethernet ports. By following the APIs and guidelines here, developers will be able to write P4 programs that are portable across many devices that are conformant to the PSA.

While parts of PSA are specific to network switches, and the "Portable NIC Architecture" differs significantly from PSA in those parts, we expect the externs defined here will be of general use across multiple P4₁₆ architectures.

The Portable Switch Architecture (PSA) Model has six programmable P4 blocks and two fixed-function blocks, as shown in [Figure 1](#). The behavior of the programmable blocks is specified using P4. The Packet buffer and Replication Engine (PRE) and the Buffer Queuing Engine (BQE) are target dependent functional blocks that may be configured for a fixed set of operations.

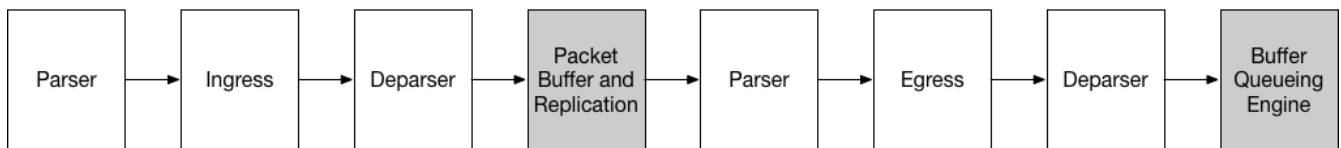


Figure 1. Portable Switch Pipeline

Incoming packets are parsed and validated, and then passed to an ingress match action pipeline, which makes decisions on where the packets go. The ingress deparser P4 code specifies the packet contents to be sent to the packet buffer, and what metadata related to the packet is carried with it. After the ingress pipeline, the packet may optionally be replicated (i.e. copies made to multiple egress ports), then stored in the packet buffer.

For each such egress port, the packet passes through an egress parser and match action pipeline before it is deparsed and queued to leave the pipeline.

A programmer targeting the PSA is required to define objects in P4 for the programmable blocks that conform to APIs defined later (see [Chapter 6](#)). The programmable block inputs and outputs are templated on user defined headers and metadata. Once these six blocks are defined, a P4 program for PSA instantiates the **main package** object, with the programmable blocks passed as arguments (see [Section 8.3](#) for an example).

A P4 programmer wishing to maximize the portability of their program should follow several general guidelines:

- Do not use undefined values in a way that affects the resulting output packet(s), or for side effects such as updating **Counter**, **Meter** or **Register** instances.
- Use as few resources as possible, e.g. table search key bits, array sizes, quantity of metadata associated with packets, etc.

This document contains excerpts of several P4₁₆ programs that use the `psa.p4` include file and demonstrate features of PSA. Source code for the complete programs can be found in the official repository containing the PSA specification{PSAExamplePrograms}.

2. Naming conventions

In this document we use the following naming conventions:

- Types are named using CamelCase followed by `_t`. For example, `PortId_t`.
- Control types and extern object types are named using CamelCase. For example `IngressParser`.
- Struct types are named using lower case words separated by `_` followed by `_t`. For example `psa_ingress_input_metadata_t`.
- Actions, extern methods, extern functions, headers, structs, and instances of controls and externs start with lower case and words are separated using `_`. For example `send_to_port`.
- Enum members, const definitions, and `#define` constants are all caps, with words separated by `_`. For example `PSA_PORT_CPU`.

Architecture specific metadata (e.g. structs) are prefixed by `psa_`.

3. Packet paths

Figure 2 shows all possible paths for packets that must be supported by a PSA implementation. An implementation is allowed to support paths for packets that are not described here.

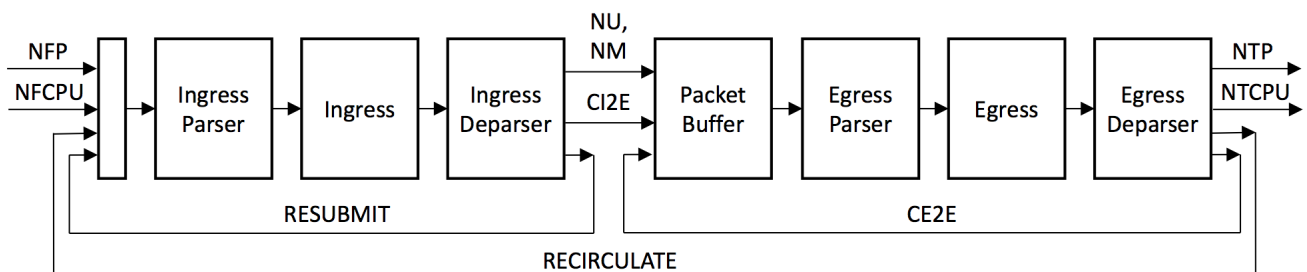


Figure 2. Packet Paths in PSA

Table 1 defines the meanings of the abbreviations in Figure 2. There can be one or more hardware, software, or PSA architecture components between the "packet source" and "packet destination" given in that table, e.g. a normal multicast packet passes through the packet replication engine and typically also a packet buffer after leaving the ingress deparser, before arriving at the egress parser. This table focuses on the P4-programmable portions of the architecture as sources and destinations of these packet paths.

Table 1. Packet path abbreviation meanings

Abbreviation	Description	Packet Source	Packet Destination
NFP	normal packet from port	port	ingress parser
NFCPU	packet from CPU port	CPU port	ingress parser
NU	normal unicast packet from ingress to egress	ingress deparser	egress parser

Abbreviation	Description	Packet Source	Packet Destination
NM	normal multicast-replicated packet from ingress to egress	ingress deparser, with help from PRE	egress parser (more than one copy is possible)
NTP	normal packet to port	egress deparser	port
NTCPU	normal packet to CPU port	egress deparser	CPU port
RESUB	resubmitted packet	ingress deparser	ingress parser
CI2E	clone from ingress to egress	ingress deparser	egress parser
RECIRC	recirculated packet	egress deparser	ingress parser
CE2E	clone from egress to egress	egress deparser	egress parser

Table {results-of-one-pkt-thru-ingress-or-egress} shows what can happen to a packet as a result of a single time being processed in ingress, or a single time being processed in egress. The cases are the same as shown in [Table 1](#), but have been grouped together by similar values of "Processed next by".

Table 2. Result of packet processed one time by ingress or egress

Abbreviation	Description	Processed next by	Resulting packet(s)
NFP	normal packet from port	ingress	At most one CI2E packet, plus at most one of a RESUB, NU, or NM packet. See Section 7.3 for details.
NFCPU	packet from CPU port		
RESUB	resubmitted packet		
RECIRC	recirculated packet		
NU	normal unicast packet from ingress to egress	egress	At most one CE2E packet, plus at most one of a RECIRC, NTP, or NTCPU packet. See Section 7.6 for details.
NM	normal multicast-replicated packet from ingress to egress		
CI2E	clone from ingress to egress		
CE2E	clone from egress to egress		
NTP	normal packet to port	device at other end of port	determined by the other device
NTCPU	normal packet to CPU port	CPU	determined by CPU

There are metadata fields defined by PSA that enable your P4 program to identify which path each packet arrived on, and to control where it will go next. See [Chapter 7](#).

For egress packets, the choice between one of multiple egress ports, the port to the CPU, or the "recirculation port", is made by the immediately previous processing step (ingress for NU, NM, or CI2E packets, egress for CE2E packets). Egress processing can choose to drop the packet instead of sending it to the port chosen earlier, but it cannot change the choice to a different port. Ingress code is the most common place in a P4 program where the output port(s) are chosen. The only exception to ingress choosing the output port is for egress-to-egress clone packets, whose destination port is chosen when the clone is created in the immediately preceding egress processing step. See [Section D.2](#) for why this

restriction exists.

A single packet received by a PSA system from a port can result in 0, 1, or many packets going out, all under control of the P4 program. For example, a single packet received from a port could cause all of the following to occur, if the P4 program so directed it:

- The original packet received as NFP from port 2. Ingress processing creates a CI2E clone destined for the CPU port (copy 1), and a multicast NM packet to multicast group 18, which is configured in the PacketReplicationEngine to have copies made to ports 5 (copy 2) and the recirculate port `PSA_PORT_RECIRCULATE` (copy 3).
- Copy 1 performs egress processing, which sends the packet on path NTCPU to the CPU port.
- Copy 2 performs egress processing, which creates a CE2E clone destined for port 8 (copy 4), and sends a NTP packet to port 5.
- Copy 3 performs egress processing, which does a RECIRC back to ingress (copy 5).
- Copy 4 performs egress processing, which sends a NTP packet to port 8.
- Copy 5 performs ingress processing, which sends a NU packet destined for port 1 (copy 6).
- Copy 6 performs egress processing, which drops the packet instead of sending it to port 1.

This is simply an example of what is possible given an appropriately written P4 program. There is no need to use all of the packet paths available. The numbering of the packet copies above is only for purposes of distinctly identifying each one in the example. The ports described in the example are also arbitrary. A PSA implementation is free to perform the steps above in many possible orders.

There is no mandated mechanism in PSA to prevent a single received packet from creating packets that continue to recirculate, resubmit, or clone from egress to egress indefinitely. This can be prevented by suitable testing of your P4 program, and/or creating in your P4 program a "time to live" metadata field that is carried along with copies of a packet, similar to the IPv4 Time To Live header field.

A PSA implementation may optionally drop resubmitted, recirculated, or egress-to-egress clone packets after an implementation-specific maximum number of times from the same original packet. If so, the implementation should maintain counters of packets dropped for these reasons, and preferably record some debug information about the first few packets dropped for these reasons (perhaps only one).

4. PSA Data types

4.1. PSA type definitions

Each PSA implementation will have specific bit widths for the following types in the data plane. These widths are defined in the target specific `psa.p4` include file. They are expected to differ from one PSA implementation to another^[1].

For each of these types, the P4Runtime API^[2] may use bit widths independent of the targets. These widths are defined by the P4Runtime API specification, and they are expected to be at least as large as the corresponding `InHeader_t` type below, such that they hold a value for any target. All PSA implementations must use data plane sizes for these types no wider than the corresponding `InHeader_t`-defined types.

```
/* These are defined using `typedef`, not `type`, so they are truly
 * just different names for the type bit<W> for the particular width W
 * shown. Unlike the `type` definitions below, values declared with
 * the `typedef` type names can be freely mingled in expressions, just
 * as any value declared with type bit<W> can. Values declared with
```

```

* one of the `type` names below _cannot_ be so freely mingled, unless
* you first cast them to the corresponding `typedef` type. While
* that may be inconvenient when you need to do arithmetic on such
* values, it is the price to pay for having all occurrences of values
* of the `type` types marked as such in the automatically generated
* control plane API.
*
* Note that the width of typedef <name>Uint_t will always be the same
* as the width of type <name>_t. */
typedef bit<unspecified> PortIdUint_t;
typedef bit<unspecified> MulticastGroupUint_t;
typedef bit<unspecified> CloneSessionIdUint_t;
typedef bit<unspecified> ClassOfServiceUint_t;
typedef bit<unspecified> PacketLengthUint_t;
typedef bit<unspecified> EgressInstanceUint_t;
typedef bit<unspecified> TimestampUint_t;

@p4runtime_translation("p4.org/psa/v1/PortId_t", 32)
type PortIdUint_t      PortId_t;
@p4runtime_translation("p4.org/psa/v1/MulticastGroup_t", 32)
type MulticastGroupUint_t MulticastGroup_t;
@p4runtime_translation("p4.org/psa/v1/CloneSessionId_t", 16)
type CloneSessionIdUint_t CloneSessionId_t;
@p4runtime_translation("p4.org/psa/v1/ClassOfService_t", 8)
type ClassOfServiceUint_t ClassOfService_t;
@p4runtime_translation("p4.org/psa/v1/PacketLength_t", 16)
type PacketLengthUint_t  PacketLength_t;
@p4runtime_translation("p4.org/psa/v1/EgressInstance_t", 16)
type EgressInstanceUint_t EgressInstance_t;
@p4runtime_translation("p4.org/psa/v1/Timestamp_t", 64)
type TimestampUint_t      Timestamp_t;
typedef error  ParserError_t;

const PortId_t PSA_PORT_RECIRCULATE = (PortId_t) unspecified;
const PortId_t PSA_PORT_CPU = (PortId_t) unspecified;

const CloneSessionId_t PSA_CLONE_SESSION_TO_CPU = (CloneSessionId_t) unspecified;

/* Note: All of the types with `InHeader` in their name are intended
* only to carry values of the corresponding types in packet headers
* between a PSA device and the P4Runtime Server software that manages
* it.
*
* The bit widths here are _independent_ of any particular PSA target
* device, and should _not_ be customized for each target.
*
* The bit widths are intended to be at least as large as any PSA
* device will ever have for that type. Thus these types may also be
* useful to define packet headers that are sent directly between a
* PSA device and other devices, without going through P4Runtime
* Server software (e.g. this could be useful for sending packets to a
* controller or data collection system using higher packet rates than
* the P4Runtime Server can handle). If used for this purpose, there
* is no requirement that the PSA data plane _automatically_ perform
* the numerical translation of these types that would occur if the
* header went through the P4Runtime Server. Any such desired
* translation is up to the author of the P4 program to perform with
* explicit code.
*
* All widths must be a multiple of 8, so that any subset of these
* fields may be used in a single P4 header definition, even on P4
* implementations that restrict headers to contain fields with a
* total length that is a multiple of 8 bits. */

```



```

/* See the comments near the definition of PortIdUint_t for why these
 * typedef definitions exist. */
typedef bit<32> PortIdInHeaderUint_t;
typedef bit<32> MulticastGroupInHeaderUint_t;
typedef bit<16> CloneSessionIdInHeaderUint_t;
typedef bit<8> ClassOfServiceInHeaderUint_t;
typedef bit<16> PacketLengthInHeaderUint_t;
typedef bit<16> EgressInstanceInHeaderUint_t;
typedef bit<64> TimestampInHeaderUint_t;

@p4runtime_translation("p4.org/psa/v1/PortIdInHeader_t", 32)
type PortIdInHeaderUint_t PortIdInHeader_t;
@p4runtime_translation("p4.org/psa/v1/MulticastGroupInHeader_t", 32)
type MulticastGroupInHeaderUint_t MulticastGroupInHeader_t;
@p4runtime_translation("p4.org/psa/v1/CloneSessionIdInHeader_t", 16)
type CloneSessionIdInHeaderUint_t CloneSessionIdInHeader_t;
@p4runtime_translation("p4.org/psa/v1/ClassOfServiceInHeader_t", 8)
type ClassOfServiceInHeaderUint_t ClassOfServiceInHeader_t;
@p4runtime_translation("p4.org/psa/v1/PacketLengthInHeader_t", 16)
type PacketLengthInHeaderUint_t PacketLengthInHeader_t;
@p4runtime_translation("p4.org/psa/v1/EgressInstanceInHeader_t", 16)
type EgressInstanceInHeaderUint_t EgressInstanceInHeader_t;
@p4runtime_translation("p4.org/psa/v1/TimestampInHeader_t", 64)
type TimestampInHeaderUint_t TimestampInHeader_t;

```

4.2. PSA supported metadata types

```

enum PSA_PacketPath_t {
    NORMAL,          /// Packet received by ingress that is none of the cases below.
    NORMAL_UNICAST,  /// Normal packet received by egress which is unicast
    NORMAL_MULTICAST, /// Normal packet received by egress which is multicast
    CLONE_I2E,        /// Packet created via a clone operation in ingress,
                      /// destined for egress
    CLONE_E2E,        /// Packet created via a clone operation in egress,
                      /// destined for egress
    RESUBMIT,         /// Packet arrival is the result of a resubmit operation
    RECIRCULATE       /// Packet arrival is the result of a recirculate operation
}

struct psa_ingress_parser_input_metadata_t {
    PortId_t          ingress_port;
    PSA_PacketPath_t  packet_path;
}

struct psa_egress_parser_input_metadata_t {
    PortId_t          egress_port;
    PSA_PacketPath_t  packet_path;
}

struct psa_ingress_input_metadata_t {
    // All of these values are initialized by the architecture before
    // the Ingress control block begins executing.
    PortId_t          ingress_port;
    PSA_PacketPath_t  packet_path;
    Timestamp_t        ingress_timestamp;
    ParserError_t      parser_error;
}

struct psa_ingress_output_metadata_t {
    // The comment after each field specifies its initial value when the
    // Ingress control block begins executing.

```

```

ClassOfService_t      class_of_service; // 0
bool                  clone;             // false
CloneSessionId_t      clone_session_id; // initial value is undefined
bool                  drop;              // true
bool                  resubmit;           // false
MulticastGroup_t      multicast_group;   // 0
PortId_t              egress_port;       // initial value is undefined
}

struct psa_egress_input_metadata_t {
    ClassOfService_t      class_of_service;
    PortId_t              egress_port;
    PSA_PacketPath_t      packet_path;
    EgressInstance_t      instance;        /// instance comes from the
PacketReplicationEngine
    Timestamp_t           egress_timestamp;
    ParserError_t         parser_error;
}

/// This struct is an 'in' parameter to the egress deparser. It
/// includes enough data for the egress deparser to distinguish
/// whether the packet should be recirculated or not.
struct psa_egress_deparser_input_metadata_t {
    PortId_t              egress_port;
}

struct psa_egress_output_metadata_t {
    // The comment after each field specifies its initial value when the
    // Egress control block begins executing.
    bool                  clone;           // false
    CloneSessionId_t      clone_session_id; // initial value is undefined
    bool                  drop;           // false
}

```

4.3. Match kinds

PSA supports the following additional `match_kind` types, over and above the three defined in the P4₁₆ language specification:

```

match_kind {
    range,      /// Used to represent min..max intervals
    selector,   /// Used for dynamic action selection via the ActionSelector extern
    optional    /// Either an exact match, or a wildcard matching any value for the entire
field
}

```

`selector` is only supported for tables with an action selector implementation. See [Section 8.12](#) for details.

4.3.1. Range tables

If a table has at least one `range` field, then a single search key could match multiple table entries. Every entry must be assigned a numeric priority by the control plane software when it is installed. If multiple installed table entries match the same search key, one among them with the maximum numeric priority will "win", and its action performed. If there are multiple matching table entries with the same maximum numeric priority, it is implementation-specific which one will have its action performed. Control plane software should assign different priority values to table entries that can match the same packet if they wish to avoid this implementation-specific behavior.

The winner is one with maximum numeric priority value if you use the P4Runtime API to specify the

numeric priorities. Check the documentation of your control plane API if you use a different one, as some APIs might choose to use the convention that smaller numeric priority values win over larger ones.

A range table may have one or more `lpm` fields. If so, the prefix length is used to determine whether a search key matches the entry, but the prefix length does *not* determine the relative priority among multiple matching table entries. Only the numeric priority supplied by the control plane software determines that.

If a range table has entries defined via a `const entries` table property, then the relative priority of the entries are highest priority first, to lowest priority last, based upon the order they appear in the P4 program.

4.3.2. Ternary tables

If a table has no `range` field, but at least one `ternary` or `optional` field, then as for tables with `range` fields, a single search key can be matched by multiple table entries, and thus every entry must have a numeric priority assigned by the control plane software. The same note about `lpm` fields in the previous section applies here, as well as the note about entries specified via `const entries`.

4.3.3. Longest prefix match tables

If a table has no `range`, `ternary`, nor `optional` fields, but at least one `lpm` field, there must be exactly one `lpm` field. There may be 0 or more `exact` fields in addition to the `lpm` field. While there can be multiple installed table entries that match a single search key, there can be at most one matching table entry of each possible prefix length of the `lpm` field (because no two table entries installed at the same time are allowed to have the same search key). The matching entry with the longest prefix length is always the winner. The control plane cannot specify a priority when installing entries for such a table—it is always determined by the prefix length.

If a longest prefix match table has entries defined via a `const entries` table property, then the relative priority of the entries are determined by the prefix lengths, not by the order they appear in the P4 program.

4.3.4. Exact match tables

If a table has only `exact` fields, then for any search key, there can be at most one matching table entry, because duplicate search keys are not allowed to be installed. Thus no numeric priority is ever needed to determine the "winning" matching table entry.

If an exact match table has entries defined via a `const entries` table property, there can be at most one matching entry for any search key, so the relative order that entries appear in the P4 program is unimportant in determining which entry will win.

[1] It is expected that ``psa.p4`` include files for different targets will typically be nearly identical to each other. Besides the possibility of differing bit widths for these PSA types, the only expected differences between ``psa.p4`` files for different targets would be annotations on externs, etc. that the P4 compiler for that target needs to do its job.

[2] The P4Runtime API is defined as a Google Protocol Buffer ``proto`` file and an accompanying English specification document here: <https://github.com/p4lang/p4runtime>

5. Data plane vs. control plane data representations

A PSA data plane may support multiple control plane APIs. Some of the notes in this section apply specifically to the case of the P4Runtime API^[2 - Chapter 4] when used to control a PSA device. If you are using a different control plane API to control a PSA device, you should consult the documentation for that control plane API to learn exactly what API it provides to configure your PSA device.

A PSA data plane implementation that supports the P4Runtime API^[2 - Chapter 4] includes software called a "P4Runtime Server" that enables runtime programming of the PSA device from one or more "P4Runtime Clients". For brevity, here we will call a P4Runtime Server an "agent", and a P4Runtime Client a "controller". A controller may control multiple devices with different PSA implementations.

As mentioned in [Section 4.1](#), different PSA implementations are expected to customize the size of the data types that refer directly to those objects in the data plane, i.e. ports, multicast group ids, etc.

Some PSA implementations are expected to use noticeably fewer resources for things like table keys and action parameters if the data plane stores only the fewest number of bits required for values of these types, for that implementation. For example, an implementation that defines `PortId_t` as `bit<6>` instead of `bit<16>`, and can take advantage of this difference, could save 10 Mbits of storage in a table with a million entries^[1].

The P4Runtime API uses quantities with bit widths independent of the target device to hold values of the types listed in [Section 4.1](#), to simplify the manipulation of these values in the controller and agent software. For control plane operations on tables, any trimming or padding of values will be performed in the agent (usually trimming in the direction from controller to device, and padding in the direction from device to controller).

There are multiple channels of communication over which such values might be carried between the controller and the data plane. These channels of communication include:

- Control plane operations on tables, where values of these types may be included as part of the key, or as an action parameter.
- Control plane operations on parser value sets, where values of these types may be included as part of the key.
- Packets sent to the CPU ("packet in" from the controller's perspective), or received from the CPU ("packet out" from the controller's perspective).
- Fields in a `Digest` extern notification message ([Section 8.14](#)).
- Fields in the data contents of a `Register` array ([Section 8.9](#)).

NOTE | There may be other channels not listed above.

For packets between the control plane and the PSA device, there is the issue that many PSA implementations are expected to restrict P4 programs to have headers with fields with a total length that is a multiple of 8 bits. To make it possible to define P4 header types that meet this restriction, and contain values of fields with these PSA-specific types, and be source-compatible across multiple PSA implementations, additional types are defined that contain `InHeader` in their name. For example, `PortIdInHeader_t` is the same as `PortId_t`, except it must be a multiple of 8 bits long, and contain at least as many bits as `PortId_t` does.

Because these `InHeader` types are guaranteed to be a multiple of 8 bits long, you may include any combination of them in a P4 header type definition, as long as the other fields in the header satisfy the

multiple of 8 bits restriction. The controller or P4 program generating packets with such headers should fill in any most significant "padding" bits with 0. You may do this with a normal assignment statement in your P4 program, where the value on the right hand side is cast to the wider `InHeader` type. Similarly, casting a value of a wider type such as `PortIdInHeader_t` to the corresponding narrower type `PortId_t` truncates the excess most significant bits as part of the cast.

Values of type `PortId_t` have an unusual property in PSA implementations. Because it can make some hardware implementations more straightforward, the numerical values of fields with type `PortId_t` in the P4 data plane might not be a simple range of values such as 0 through 31, as one might choose when writing control plane software for a 32-port device. The agent is expected to implement numerical translation between controller port id values and data plane port id values, for each of the channels of communication between the controller and data plane described above.

The file `psa.p4` contains an annotation `p4runtime_translation` on the type definition of types `PortId_t` and `PortIdInHeader_t`. This enables the compiler to mark all uses of values of these types that are accessible from the P4Runtime API, so the agent software knows that it must translate them, and what kind of translation to perform. The benefit is that you do not need to put any special markings on your uses of values of these types throughout your P4 program.

The cost of this approach is: if you want to do arithmetic on values of these types, you must explicitly cast them to a `bit<W>` type. The `psa.p4` include file defines `PortIdUInt_t` as a `typedef` with exactly the same width in bits as type `PortId_t`, so you can cast values of type `PortId_t` to type `PortIdUInt_t`, and then you can perform all P4 arithmetic operations on the value. The result must be explicitly cast back to type `PortId_t` if you wish to assign it to a metadata field with that type. Corresponding types with `UInt` in their name are defined for all PSA types.

Because of this translation, we recommend that values of type `PortId_t` be treated similarly to values of an `enum` type. Comparing two values of this type for equality or "not equal to" is reasonable, as well as assigning the values to other variables or parameters of the same type, but nearly any other operations are prone to error. When matching values of type `PortId_t` as part of a table key, always match a complete value exactly, or wildcard every bit of the value (i.e. a `ternary` match kind with all bits wildcard, or an `lpm` match kind with prefix length 0). If you attempt to do any of the following things on a value with type `PortId_t` or `PortIdInHeader_t`, the numerical translation performed may lead to functional errors in your program:

- Do a table key match on a subset of the bits, or a range match.
- Compare port values with relational operators like `<` or `>`.
- Compare port values to specific numeric literal values like 0 or 0xff. It is recommended instead to compare their values by using them as table key fields, or parser value set key fields, against values installed by the control plane (which have been translated to the corresponding device-specific value as determined by the device's agent software). It is also reasonable to compare port values for equality against the symbolic constant values `PSA_PORT_CPU` or `PSA_PORT_RECIRCULATE`, which have target-specific numeric values.
- Perform arithmetic on the value, and expect to get a value that corresponds to another port of the device. Some numerical values may not correspond to any port of the device, and the values corresponding to ports need not be consecutive.

The list above is not intended to be exhaustive.

All of the comments above apply to all types where numerical translation occurs between the controller and the data plane. Below is a complete list of numeric types planned for numerical translation by default in PSA:

- `PortId_t` or `PortIdInHeader_t`
- `ClassOfService_t` or `ClassOfServiceInHeader_t`

For the types listed below, no numerical translation occurs by default^[2]. A PSA data plane must support all numerical values from 0 up to the maximum value that it supports. Except for `Timestamp_t` values, the number of values supported by the data plane need not be a power of 2. Controllers must have a way to discover each PSA device's maximum supported value for each of these types.

- `MulticastGroup_t` - 0 is a special value that indicates no multicast replication is to be performed for a packet, so this type is an exception to the rule above that 0 must be supported in the data plane.
- `CloneSessionId_t`
- `PacketLength_t`
- `EgressInstance_t`
- `Timestamp_t`

NOTE	TBD: Values of type <code>Timestamp_t</code> are being considered for numerical translation in the agent software, between target-specific values, and a value with a common unit and 0 value reference across all targets.
-------------	---

Note that all of these types have a `p4runtime_translation` annotation in the `psa.p4` include file. This is to ensure that when the compiler generates a P4Runtime P4Info file from source programs, it will include in the P4Info file the type specified by the `p4runtime_translation` rather than the target-specific bit width. For the same P4 source program, the P4Info file contents are intended to be identical for all targets.

If the type bitwidth specified as the second parameter to the `p4runtime_translation` is different from the device-specific bitwidth (of the underlying type), we expect the P4Runtime server to perform the appropriate casting. Additionally, more advanced numerical translation can be enabled at runtime for any type annotated with `p4runtime_translation`, although arbitrary numerical translation is only mandated for `PortId_t`, `ClassOfService_t`, and their `Inheader` variants. To request arbitrary numerical translation for a give type, the P4Runtime system will expect the URI (first parameter to the `p4runtime_translation`) and the desired mapping.

[1] While 10 Mbits sounds tiny to one accustomed to computers with hundreds of gigabytes of DRAM, the highest speed PSA implementations are ASICs that must keep tables in on-chip memories, similar to caches in general purpose CPUs. The Intel i9-7980XE released in 2017 has 198 Mbits of on-chip L3 cache shared by its CPU cores. Among Intel processors in Intel's 7th generation Core released in 2017 with at least 100 Mbits of L3 cache, they all cost close to \$9 per Mbit of L3 cache. https://en.wikipedia.org/wiki/List_of_Intel_microprocessors

[2] The open source `p4c` P4 compiler is planned to support an option to enable numerical translation for additional types, without modifying your P4 program, nor the `psa.p4` include file. These additional types would be specified by their name.

6. Programmable blocks

The following declarations provide a template for the programmable blocks in the PSA. The P4 programmer is responsible for implementing controls that match these interfaces and instantiate them in a package definition.

It uses the same user-defined metadata type `IM` and header type `IH` for all ingress parsers and control blocks. The egress parser and control blocks can use the same types for those things, or different types, as the P4 program author wishes.

```
parser IngressParser<H, M, RESUBM, RECIRCM>(  
    packet_in buffer,  
    out H parsed_hdr,  
    inout M user_meta,  
    in psa_ingress_parser_input_metadata_t istd,  
    in RESUBM resubmit_meta,  
    in RECIRCM recirculate_meta);  
  
control Ingress<H, M>(  
    inout H hdr, inout M user_meta,  
    in psa_ingress_input_metadata_t istd,  
    inout psa_ingress_output_metadata_t ostd);  
  
control IngressDeparser<H, M, CI2EM, RESUBM, NM>(  
    packet_out buffer,  
    out CI2EM clone_i2e_meta,  
    out RESUBM resubmit_meta,  
    out NM normal_meta,  
    inout H hdr,  
    in M meta,  
    in psa_ingress_output_metadata_t istd);  
  
parser EgressParser<H, M, NM, CI2EM, CE2EM>(  
    packet_in buffer,  
    out H parsed_hdr,  
    inout M user_meta,  
    in psa_egress_parser_input_metadata_t istd,  
    in NM normal_meta,  
    in CI2EM clone_i2e_meta,  
    in CE2EM clone_e2e_meta);  
  
control Egress<H, M>(  
    inout H hdr, inout M user_meta,  
    in psa_egress_input_metadata_t istd,  
    inout psa_egress_output_metadata_t ostd);  
  
control EgressDeparser<H, M, CE2EM, RECIRCM>(  
    packet_out buffer,  
    out CE2EM clone_e2e_meta,  
    out RECIRCM recirculate_meta,  
    inout H hdr,  
    in M meta,  
    in psa_egress_output_metadata_t istd,  
    in psa_egress_deparser_input_metadata_t edstd);  
  
package IngressPipeline<IH, IM, NM, CI2EM, RESUBM, RECIRCM>(  
    IngressParser<IH, IM, RESUBM, RECIRCM> ip,  
    Ingress<IH, IM> ig,  
    IngressDeparser<IH, IM, CI2EM, RESUBM, NM> id);
```



```

package EgressPipeline<EH, EM, NM, CI2EM, CE2EM, RECIRC>(  
    EgressParser<EH, EM, NM, CI2EM, CE2EM> ep,  
    Egress<EH, EM> eg,  
    EgressDeparser<EH, EM, CE2EM, RECIRC> ed);  
  
package PSA_Switch<IH, IM, EH, EM, NM, CI2EM, CE2EM, RESUBM, RECIRC> (  
    IngressPipeline<IH, IM, NM, CI2EM, RESUBM, RECIRC> ingress,  
    PacketReplicationEngine pre,  
    EgressPipeline<EH, EM, NM, CI2EM, CE2EM, RECIRC> egress,  
    BufferingQueueingEngine bqe);

```

7. Packet Path Details

Refer to section {packet-paths} for the packet paths provided by PSA, and their abbreviated names, used often in this section.

7.1. Initial values of packets processed by ingress

Table [#ingress-initial-values] describes the initial values of the packet contents and metadata when a packet begins ingress processing.

Table 3. Initial values for packets processed by ingress

	NFP	NFCPU	RESUB	RECIRC
packet_in	see text			
user_meta	see text			
IngressParser istd fields (type psa_ingress_parser_input_metadata_t)				
ingress_port	PortId_t value of packets' input port	PSA_PORT_CPU	copied from resub'd packet	PSA_PORT_RECIRCULATE
packet_path	NORMAL	NORMAL	RESUBMIT	RECIRCULATE
Ingress istd fields (type psa_ingress_input_metadata_t)				
ingress_port	Same value as received by IngressParser above.			
packet_path	Same value as received by IngressParser above.			
ingress_timestamp	Time that packet began processing in IngressParser. For RESUB or RECIRC packets, the time the 'copy' began IngressParser, not the original.			
parser_error	From IngressParser. Always error.NoError if there was no parser error.			

Note that the ingress_port value for a resubmitted packet could be PSA_PORT_RECIRCULATE if a packet was recirculated, and then that recirculated packet was resubmitted.

7.2. Initial packet contents for packets from ports

For Ethernet ports, packet_in for FP and NFCPU path packets contains the Ethernet frame starting with the Ethernet header. It does not include the Ethernet frame CRC.

NOTE

TBD Whether the payload is always the minimum of 46 bytes (64 byte minimum Ethernet frame size, minus 14 bytes of header, minus 4 bytes of CRC), or whether an

implementation is allowed to leave some of those bytes out.

The PSA does not put further restrictions on `packet_in.length()` as defined in the P4₁₆ spec. Targets that do not support it, should provide mechanisms to raise an error.

The P4Runtime has a "Packet Out" capability to send a packet from the controller to a PSA device. Such packets are sent into the PSA device as NFCPU path packets. There is no metadata associated with such packets, only the contents of the packet that are parsed normally by the P4 program's IngressParser code. There may be some translation of header field values, as described in [Chapter 5](#).

7.2.1. Initial packet contents for resubmitted packets

For RESUB packets, `packet_in` is the same as the pre-IngressParser contents of `packet_in`, for the packet that caused this resubmitted packet to occur (i.e. with NO modifications from any ingress processing).

7.2.2. Initial packet contents for recirculated packets

For RECIRC packets, `packet_in` is created by starting with the headers emitted by the egress deparser of the egress packet that was recirculated, followed by the payload of that packet, i.e. the part that was not parsed by the egress parser.

7.2.3. User-defined metadata for all ingress packets

The PSA architecture does not mandate initialization of user-defined metadata to known values as given as input to the ingress parser. If a user's P4 program explicitly initializes all user-defined metadata early on (e.g. in the parser's `start` state), then that will flow through the rest of the parser into the `Ingress` control block as one might normally expect. This will be left as an option to the user in their P4 programs, not required behavior for all P4 programs.

There are two direction `in` parameters to the ingress parser with user-defined types, named `resubmit_meta` and `recirculate_meta`. They may be used to carry metadata for resubmitted and recirculated packets.

Consider a packet that arrives at the ingress pipeline, and during ingress processing the P4 program assigns values to fields of the PSA standard metadata such that the packet is resubmitted (see [Section 7.3](#) for details on how to do so). In the ingress deparser, the P4 program assigns a value to the `out` parameter named `resubmit_meta`. This value (which can be a collection of many individual values in fields, sub-structs, headers, etc.) becomes associated with the resubmitted packet by the PSA implementation, and when the resubmitted packet begins ingress parsing, that becomes the value of the `in` parameter named `resubmit_meta` to the ingress parser.

For resubmitted packets, the value of the `in` parameter named `recirculate_meta` is undefined.

Conversely, for recirculated packets, the value of the `in` parameter named `recirculate_meta` contains whatever value was assigned to the egress deparser `out` parameter named `recirculate_meta` when the packet was recirculated. The value of the `in` parameter `resubmit_meta` is undefined for recirculated packets.

For packets from a port, including the CPU port, both of the `in` parameters `resubmit_meta` and `recirculate_meta` are undefined.

7.3. Behavior of packets after ingress processing is complete

The pseudocode below defines where copies of packets will be made after the `Ingress` control block has completed executing, based upon the contents of several metadata fields in the struct `psa_ingress_output_metadata_t`.

The function `platform_port_valid()` mentioned below takes a value of type `PortId_t`, returning `true` only when the value represents an output port for the implementation. It is expected that for some PSA implementations there will be bit patterns for a value of type `PortId_t` that do not correspond to any port. This function returns true for both `PSA_PORT_CPU` and `PSA_PORT_RECIRCULATE`. `platform_port_valid` is not defined in PSA for calling from the P4 data-plane program, since there is no known use case for calling it at packet processing time. It is intended for describing the behavior in pseudocode. The control plane is expected to configure tables with valid port numbers.

A comment saying "recommended to log error" is not a requirement, but a recommendation, that a PSA implementation should maintain a counter that counts the number of times this error occurs. It would also be useful if the implementation recorded details about the first few times this error occurred, e.g. a FIFO queue of the first several invalid values of `ostd.egress_port` that cause an error to occur, perhaps with other information about the packet that caused it, with tail dropping if it fills up. Control plane or driver software would be able to read these counters, and read and drain the FIFO queues to assist P4 developers in debugging their code.

```
struct psa_ingress_output_metadata_t {
    // The comment after each field specifies its initial value when the
    // Ingress control block begins executing.
    ClassOfService_t    class_of_service; // 0
    bool                clone;             // false
    CloneSessionId_t    clone_session_id; // initial value is undefined
    bool                drop;              // true
    bool                resubmit;           // false
    MulticastGroup_t    multicast_group;   // 0
    PortId_t            egress_port;       // initial value is undefined
}
```

First we give an outline of behavior, for quick reference of the relative priority of the possible actions. This outline is only for reader convenience—it is not the specification for the behavior.

```
psa_ingress_output_metadata_t ostd;

if (ostd.clone) {
    create ingress to egress clone(s), with options as configured by
    the PRE clone session numbered ostd.clone_session_id;
} else { no clone; }

if (ostd.drop) { drop packet; }
else if (ostd.resubmit) { resubmit packet; }
else if (ostd.multicast_group != 0) { PRE multicast replicates packet; }
else { PRE sends one copy of packet to ostd.egress_port; }
```

The pseudocode below defines the behavior a PSA implementation must follow.

```
psa_ingress_output_metadata_t ostd;
```

```

if (ostd.clone) {
    if (ostd.clone_session_id value is supported) {
        from the values configured for ostd.clone_session_id in PRE {
            cos = class_of_service
            set((egress_port[0], instance[0]), ..., (egress_port[n], instance[n])) =
                set of egress_port and instance pairs
            trunc = truncate
            plen = packet_length_bytes
        }
        if (cos value is not supported) {
            cos = 0;
            // Recommended to log error about unsupported cos value.
        }
        for each pair (egress_port, instance) in the set {
            Create a clone of the packet and send it to the packet
            buffer with the egress_port, instance, and
            class_of_service cos, after which it will start egress
            processing. It will contain at most the first plen
            bytes of the packet as received at the ingress parser
            if trunc is true, otherwise the entire packet.
        }
    } else {
        // Do not create a clone. Recommended to log error about
        // unsupported ostd.clone_session_id value.
    }
}
// Continue below, regardless of whether a clone was created.
// Any clone created above is unaffected by the code below.
if (ostd.drop) {
    drop the packet
    return; // Do not continue below.
}
if (ostd.class_of_service value is not supported) {
    ostd.class_of_service = 0; // use default class 0 instead
    // Recommended to log error about unsupported
    // ostd.class_of_service value.
}
if (ostd.resubmit) {
    resubmit the packet, i.e. it will go back to starting with the
    ingress parser;
    return; // Do not continue below.
}
if (ostd.multicast_group != 0) {
    Make 0 or more copies of the packet according to the control
    plane configuration of multicast group ostd.multicast_group.
    Every copy will have the same value of ostd.class_of_service
    return; // Do not continue below.
}
if (platform_port_valid(ostd.egress_port)) {
    enqueue one packet for output port ostd.egress_port with class
    of service ostd.class_of_service
} else {
    drop the packet
    // Recommended to log error about unsupported ostd.egress_port value.
}

```

Whenever the pseudocode above indicates that a packet should be sent on a particular packet path, a PSA implementation may under some circumstances instead drop the packet. For example, the packet buffer may be too low on available space for storing new packets, or some other congestion control mechanism such as RED (Random Early Detection) or AFD (Approximate Fair Dropping) may select the packet for dropping. It is recommended that an implementation maintain counters of packets dropped, preferably with separate counters for as many different reasons as the implementation has for dropping

packets outside the control of the P4 program.

A PSA implementation may implement multiple classes of service for packets sent to the packet buffer. If so, the **Ingress** control block may choose to assign a value to the `ostd.class_of_service` field to change the packet's class of service to a value other than the default of 0.

PSA only specifies how the **Ingress** control block can control the class of service of packets. PSA does not mandate a scheduling policy among queues that may exist in the packet buffer. Something at least as flexible as weighted fair queuing, with an optional strict high priority queue, is recommended for PSA implementations with separate queues for each class of service. See [Appendix F](#) for more on packet ordering recommendations in PSA devices.

The P4Runtime API specification defines how a controller may discover the number of distinct class of service values that a PSA device supports.

7.3.1. Multicast replication

The control plane may configure each **multicast_group** in the PRE to create the desired copies of packets sent to that group. Each group begins empty. Sending a packet to an empty group causes the packet to be dropped. The control plane may add one or more pairs of the form **(egress_port, instance)** to a multicast group, and may also remove pairs from a group that were added earlier.

Suppose a multicast group contains the following set of pairs:

```
(egress_port[0], instance[0]),  
(egress_port[1], instance[1]),  
...,  
(egress_port[N-1], instance[N-1])
```

When a packet is sent to that group, **N** copies of the packet are made. Copy number **i** that is sent to egress processing will have its **struct** of type `psa_egress_input_metadata_t` filled in with the field `egress_port` equal to `egress_port[i]`, and the field `instance` filled in with `instance[i]`. Note: A multicast group is a set of pairs, and it is not required that an implementation create copies in an order that the control plane can enforce. See [Appendix F](#) for more on packet ordering recommendations in PSA devices.

Within a single multicast group, the pairs **(egress_port, instance)** must be different from each other, but it is allowed for any number of pairs within a multicast group to have the same value of `egress_port`, or to have the same value of `instance`. The same pair **(egress_port, instance)** can occur in any number of different multicast groups.

A PSA implementation need only support `egress_port` values that represent single ports of the PSA device. That is, it need not implement support for `egress_port` values that represent an entire Link Aggregation Group (LAG) interface, which is a set of physical ports over which load balancing of traffic is performed.

A PSA device must support `egress_port` values in a multicast group that are equal to `PSA_PORT_CPU` or `PSA_PORT_RECIRCULATE`. The copies of a multicast packet made to those ports will behave the same in egress as a unicast packets sent to the corresponding port, i.e. if not dropped, those copies will go to the CPU port, or be recirculated back to ingress.

7.4. Actions for directing packets during ingress

All of these actions modify one or more metadata fields in the struct with type

`psa_ingress_output_metadata_t` that is an `inout` parameter of the `Ingress` control block. None of these actions have any other immediate effect. What happens to the packet is determined by the value of all fields in that struct when ingress processing is complete, not at the time one of these actions is called. See [Section 7.3](#).

These actions are provided for convenience in making changes to these metadata fields. Their effects are expected to be common kinds of changes one will want to make in a P4 program. If they do not suit your use cases, you may modify the metadata fields directly in your P4 programs however you prefer, e.g. within actions you define.

7.4.1. Unicast operation

Sends packet to a port. See Table [#egress-initial-values], column NU, for how metadata fields are filled in when such a packet begins egress processing.

```
/// Modify ingress output metadata to cause one packet to be sent to
/// egress processing, and then to the output port egress_port.
/// (Egress processing may choose to drop the packet instead.)

/// This action does not change whether a clone or resubmit operation
/// will occur.

@noWarn("unused")
action send_to_port(inout psa_ingress_output_metadata_t meta,
                   in PortId_t egress_port)
{
    meta.drop = false;
    meta.multicast_group = (MulticastGroup_t) 0;
    meta.egress_port = egress_port;
}
```

7.4.2. Multicast operation

Sends packet to a multicast group or a port. See Table [#egress-initial-values], column NM, for how metadata fields are filled in when each multicast-replicated copy of such a packet begins egress processing.

The `multicast_group` parameter is the multicast group id. The control plane must configure the multicast groups through a separate mechanism such as the P4Runtime API.

```
/// Modify ingress output metadata to cause 0 or more copies of the
/// packet to be sent to egress processing.

/// This action does not change whether a clone or resubmit operation
/// will occur.

@noWarn("unused")
action multicast(inout psa_ingress_output_metadata_t meta,
                in MulticastGroup_t multicast_group)
{
    meta.drop = false;
    meta.multicast_group = multicast_group;
}
```

7.4.3. Drop operation

Do not send a copy of the packet for normal egress processing.

```
/// Modify ingress output metadata to cause no packet to be sent for
/// normal egress processing.

/// This action does not change whether a clone will occur. It will
/// prevent a packet from being resubmitted.

@noWarn("unused")
action ingress_drop(inout psa_ingress_output_metadata_t meta)
{
    meta.drop = true;
}
```

7.5. Initial values of packets processed by egress

Table {egress-initial-values} describes the initial values of the packet contents and metadata when a packet begins egress processing.

Table 4. Initial values for packets processed by egress

	NU	NM	CI2E	CE2E
packet_in	see text			
user_meta	see text			
EgressParser istd fields (type psa_egress_parser_input_metadata_t)				
egress_port	ostd.egress_port of ingress packet	from PRE configuration of multicast group	from PRE configuration of clone session	
packet_path	NORMAL_UNICAST	NORMAL_MULTICAST	CLONE_I2E	CLONE_E2E
Egress istd fields (type psa_egress_input_metadata_t)				
class_of_service	ostd.class_of_service of ingress packet		from PRE configuration of clone session	
egress_port	Same value as received by EgressParser above.			
packet_path	Same value as received by EgressParser above.			
instance	0	from PRE configuration of multicast group	from PRE configuration of clone session	
egress_timestamp	Time that packet began processing in EgressParser. Filled in independently for each copy of a multicast-replicated packet.			
parser_error	From EgressParser. Always error.NoError if there was no parser error. See "Multicast copies" section.			

Note that while some P4 architectures have a standard metadata field that gives the length of the packet in bytes, there is no such field in PSA. If a target device supports cut-through switching^[1] and this feature is enabled, it is possible for egress processing to begin before the last byte of the packet has arrived on the input port. Thus it is not possible for the target device to determine the packet length yet. It is recommended that a P4 developer use fields available in some packet headers to determine the length of

a packet, such as the Total Length field in the IPv4 header, or the Payload length field in the IPv6 header.

7.5.1. Initial packet contents for normal packets

For NU and NM packets, `packet_in` comes from the ingress packet that caused this packet to be sent to egress. It starts with the packet headers as emitted by the ingress deparser, followed by the payload of that packet, i.e. the part that was not parsed by the ingress parser.

Packets to be recirculated, i.e. those sent to port `PSA_PORT_RECIRCULATE` via the normal unicast or multicast packet paths, fit into this category. They are not treated differently by a PSA implementation from normal unicast or multicast packets until they reach the egress deparser.

7.5.2. Initial packet contents for packets cloned from ingress to egress

For CI2E packets, `packet_in` is from the ingress packet that caused this clone to be created. It is the same as the pre-IngressParser contents of `packet_in` of that ingress packet, with no modifications from any ingress processing. Truncation of the payload is supported.

Packets cloned in ingress using a clone session configured with `egress_port` equal to `PSA_PORT_RECIRCULATE` fit into this category.

7.5.3. Initial packet contents for packets cloned from egress to egress

For CE2E packets, `packet_in` is from the egress packet that caused this clone to be created. It starts with the headers emitted by the egress deparser, followed by the payload of that packet, i.e. the part that was not parsed by the egress parser. Truncation of the payload is supported.

Packets cloned in egress using a clone session configured with `egress_port` equal to `PSA_PORT_RECIRCULATE` fit into this category.

7.5.4. User-defined metadata for all egress packets

This is very similar to how metadata is initialized for ingress packets. See [Section 7.2.3](#).

The primary differences for egress packets are the different packet paths involved. There are three parameters with direction `in` for the egress parser, named `normal_meta`, `clone_i2e_meta`, and `clone_e2e_meta`. For every packet that begins egress processing, exactly one of those three has defined contents, and the other two have undefined contents.

For NU and NM packets, the parameter `normal_meta` is the only one with defined contents. Its value is the one that was assigned to the `out` parameter with the same name of the ingress deparser, when the normal packet was completing its ingress processing.

For CLONE_I2E packets, the parameter `clone_i2e_meta` is the only one with defined contents. Its value is the one that was assigned to the `out` parameter with the same name of the ingress deparser, when the clone was created.

For CLONE_E2E packets, the parameter `clone_e2e_meta` is the only one with defined contents. Its value is the one that was assigned to the `out` parameter with the same name of the egress deparser, when the clone was created.

7.5.5. Multicast and clone copies

The following fields may differ among copies of a multicast-replicated packet that are processed in egress. Similarly for copies of a cloned packet when they are processed in egress. Both are referred to as replicated packets in this section.

- **egress_port** - This field will typically differ among copies of a replicated packet, but it may also be the same for arbitrary copies, as determined by the control plane configuration of the PacketReplicationEngine. It is expected that the control plane will configure the PacketReplicationEngine so that each copy of the same original packet is assigned a unique value of the pair (**egress_port**, **instance**).
- **instance** - See **egress_port**
- **egress_timestamp** - This value is filled in independently for each copy of a replicated packet. Depending upon the quantity of traffic destined to each output port, the timestamp could vary significantly between copies of the same original packet.
- **parser_error** - In the common case, this will typically be the same for every copy of the same original replicated packet. However, it is determined by the EgressParser P4 code for each copy independently, so if that parsing behavior depends upon a field that can differ among copies, e.g. **egress_port**, then **parser_error** can differ among copies.

All contents of a packet and its associated metadata, other than those mentioned above, will be the same for every copy of the same original replicated packet.

7.6. Behavior of packets after egress processing is complete

The pseudocode below defines where copies of packets will be made after the **Egress** control block has completed executing, based upon the contents of several metadata fields in the struct **psa_egress_output_metadata_t**.

```
struct psa_egress_output_metadata_t {
    // The comment after each field specifies its initial value when the
    // Egress control block begins executing.
    bool clone; // false
    CloneSessionId_t clone_session_id; // initial value is undefined
    bool drop; // false
}

psa_egress_input_metadata_t istd;
psa_egress_output_metadata_t ostd;

if (ostd.clone) {
    if (ostd.clone_session_id value is supported) {
        from the values configured for ostd.clone_session_id in PRE {
            cos = class_of_service
            set((egress_port[0], instance[0]), ..., (egress_port[n], instance[n])) =
                set of egress_port and instance pairs
            trunc = truncate
            plen = packet_length_bytes
        }
        if (cos value is not supported) {
            cos = 0;
            // Recommended to log error about unsupported cos
            // value.
        }
    }
}
```

```

    }
    for each pair (egress_port, instance) in the set {
        Create a clone of the packet and send it to the packet
        buffer with the egress_port, instance, and
        class_of_service cos, after which it will start egress
        processing. It will contain at most the first plen
        bytes of the packet as sent out from the egress
        deparser if trunc is true, otherwise the entire
        packet.
    }
} else {
    // Do not create a clone. Recommended to log error about
    // unsupported ostd.clone_session_id value.
}
}
// Continue below, regardless of whether a clone was created.
// Any clone created above is unaffected by the code below.
if (ostd.drop) {
    drop the packet
    return; // Do not continue below.
}
// The value istd.egress_port below is the same one that the
// packet began its egress processing with, as decided during
// ingress processing for this packet (or as determined by the PRE
// configuration of a clone session, for cloned packets,
// regardless of whether the clone operation was done in ingress
// or egress). The egress code is not allowed to change it.
if (istd.egress_port == PSA_PORT_RECIRCULATE) {
    recirculate the packet, i.e. it will go back to starting with the
    ingress parser;
    return; // Do not continue below.
}
enqueue one packet for output port istd.egress_port

```

As for the handling of a packet after ingress processing, a PSA implementation may drop a packet after egress processing, even if the pseudocode above says that a packet will be sent. For example, you may attempt to clone a packet after egress when the packet buffer is too full, or you may attempt to recirculate a packet when the ingress pipeline is busy handling other packets. It is recommended that an implementation maintain counters of packets dropped, preferably with separate counters for as many different reasons as the implementation has for dropping packets outside the control of the P4 program.

7.7. Actions for directing packets during egress

7.7.1. Drop operation

Do not send the packet out of the device after egress processing is complete.

```

/// Modify egress output metadata to cause no packet to be sent out of
/// the device.

/// This action does not change whether a clone will occur.

@noWarn("unused")
action egress_drop(inout psa_egress_output_metadata_t meta)
{
    meta.drop = true;
}

```

7.8. Contents of packets sent out to ports

There is no metadata associated with NTP and NTCPU packets.

They begin with the series of bytes emitted by the egress deparser. Following that is the payload, which are those packet bytes that were not parsed in the egress parser.

For Ethernet ports, any padding required to get the packet up to the minimum frame size required is done by the implementation, as well as calculation of and appending the Ethernet frame CRC.

It is expected that typical P4 programs will have explicit checks to avoid sending packets larger than a port's maximum frame size. A typical implementation will drop frames larger than this maximum supported size. It is recommended that they maintain error counters for such dropped frames.

The P4Runtime has a "Packet In" capability to receive packets sent by a PSA device to the port `PSA_PORT_CPU`. There is no metadata associated with such packets, only the contents of the packet that are emitted normally by the P4 program's EgressDeparser code. There may be some translation of header field values, as described in [Section 4.1](#).

7.9. Packet Cloning

Packet cloning is a mechanism to send a copy of a packet to a specified port, in addition to the 'regular' packet. Multiple clones can be made via a single clone operation, by appropriate control plane configuration.

One use case for cloning is packet mirroring, i.e. send the packet to its normal destination according to other features implemented by the P4 program, and in addition, send a copy of the packet as received to another output port, e.g. to a monitoring device.

Packet cloning happens at the end of the ingress and/or egress pipeline. PSA specifies the following semantics for the clone operation. When the clone operation is invoked at the end of the ingress pipeline, each cloned packet is a copy of the packet as it entered the ingress parser. When the clone operation is invoked at the end of the egress pipeline, each cloned packet is a copy of the modified packet after egress processing, as output by the egress deparser. In both cases, the cloned packets are submitted to the egress pipeline for further processing.

Logically, PRE implements the mechanics of copying a packet. The metadata fields that control cloning are those whose names begin with `clone` in types `psa_ingress_output_metadata_t` and `psa_egress_output_metadata_t`.

```
bool                clone;
CloneSessionId_t    clone_session_id;
```

The `clone` flag specifies whether a packet should be cloned. If true, then a cloned packet, or packets, should be generated at the end of the pipeline. The `clone_session_id` specifies one of several possible clone sessions that the control plane may configure in the PRE. For each clone session, the control plane may configure the following values that should be associated with packets cloned using that session.

```
/// Each clone session may configure zero or more pairs of (egress_port, instance).
PortId_t          egress_port; /// egress_port in a pair of (egress_port, instance)
EgressInstance_t  instance;    /// instance in a pair of (egress_port, instance)

/// Each clone session has configuration for exactly one of each of
/// the following values.
ClassOfService_t  class_of_service;
```

```
bool          truncate;
PacketLength_t packet_length_bytes;  /// only used if truncate is true
```

The configuration of the set of ([egress_port](#), [instance](#)) values for a clone session is similar to, and has the same requirements and restrictions as, the configuration of a set of pairs for a multicast group, as described in [Section 7.3.1](#).

The [egress_port](#) values may be any ports that can be used for normal unicast packets, i.e. any normal port, [PSA_PORT_CPU](#), or [PSA_PORT_RECIRCULATE](#). For the latter two values, the cloned packet will be sent to the CPU, or recirculated at the end of egress processing, as a normal unicast packet would at the end of egress processing.

Truncation of cloned packets is supported as an optimization to reduce the bandwidth required to send the beginning of packets. This is sometimes useful in sending packet headers to the control plane, or some kinds of data collection system for traffic monitoring. Here by "headers" we simply mean "some number of bytes from the beginning of the packet", not headers as defined and parsed in your P4 program.

If [truncate](#) is false for a clone session, then no truncation is performed for packets cloned using that session.

Otherwise, packets are truncated to contain at most the first [packet_length_bytes](#) bytes of the packet, with any additional bytes removed. Truncating a packet has no effect on any metadata that is carried along with it, and the size of that metadata is not counted as part of the [packet_length_bytes](#) quantity. Any truncation is based completely upon the length of the packet as passed to the type [packet_in](#) parameter to the ingress parser (for ingress to egress clones), or as sent out as the type [packet_out](#) parameter from the egress deparser (for egress to egress clones).

PSA implementations are allowed to support only a restricted set of possible values for [packet_length_bytes](#), e.g. an implementation might choose only to support values that are multiples of 32 bytes.

Since it is an expected common case to clone packets to the CPU, every PSA implementation begins with a clone session [PSA_CLONE_SESSION_TO_CPU](#) initialized with the set of ([egress_port](#), [instance](#)) values containing exactly one pair with [egress_port](#) = [PSA_PORT_CPU](#) and [instance](#) = 0. This clone session is also initialized with the configuration values [class_of_service](#) = 0, and [truncate](#) = false.

7.9.1. Clone Examples

The partial program below demonstrates how to clone a packet.

```
header clone_i2e_metadata_t {
    bit<8> custom_tag;
    EthernetAddress srcAddr;
}
control ingress(inout headers hdr,
               inout metadata user_meta,
               in  psa_ingress_input_metadata_t istd,
               inout psa_ingress_output_metadata_t ostd)
{
    action do_clone (CloneSessionId_t session_id) {
        ostd.clone = true;
        ostd.clone_session_id = session_id;
        user_meta.custom_clone_id = 1;
    }
    table t {
```

```

    key = {
        user_meta.fwd_metadata.outport : exact;
    }
    actions = { do_clone; }
}

apply {
    t.apply();
}

}

control IngressDeparserImpl(
    packet_out packet,
    out clone_i2e_metadata_t clone_i2e_meta,
    out empty_metadata_t resubmit_meta,
    out metadata normal_meta,
    inout headers hdr,
    in metadata meta,
    in psa_ingress_output_metadata_t istd)
{
    DeparserImpl() common_deparser;
    apply {
        // Assignments to the out parameter clone_i2e_meta must be
        // guarded by this if condition:
        if (psa_clone_i2e(istd)) {
            clone_i2e_meta.custom_tag = (bit<8>) meta.custom_clone_id;
            if (meta.custom_clone_id == 1) {
                clone_i2e_meta.srcAddr = hdr.ethernet.srcAddr;
            }
        }
        common_deparser.apply(packet, hdr);
    }
}
}

```

7.10. Packet Resubmission

Packet resubmission is a mechanism to repeat ingress processing on a packet.

Packet resubmission happens at the end of the ingress pipeline. When a packet is resubmitted, the packet finishes the ingress pipeline processing and re-enters the ingress parser without being deparsed. In other words, the resubmitted packet has the same header and payload as the original packet. The `ingress_port` of the resubmitted packet is the same as the original packet. The `packet_path` of the resubmitted packet is changed to `RESUBMIT`.

The ingress parser distinguishes the resubmitted packet from the original packet with the `packet_path` field in `ingress_parser_intrinsic_metadata_t`. The ingress parser can choose a different algorithm to parse the resubmitted packet. Similarly, the ingress pipeline can choose to process the resubmitted packet with different actions as opposed to the ones used to process the original packet. Further, if a target permits the same packet to be resubmitted multiple times, the user program can distinguish the packet resubmitted the first time, or second time, by the extra metadata associated with the packet. Note the maximum number of packet resubmission for a single packet is target-dependent. See section {packet-paths}.

PSA specifies that the resubmit operation can only be used in the ingress pipeline. The egress pipeline cannot resubmit packets. As described in Section [#packet-paths], there is no mandated mechanism in PSA to prevent a single received packet from creating packets that continue to recirculate, resubmit, or clone from egress to egress indefinitely. However, targets may impose limits on the number of resubmissions, recirculations, or clones.

One use case of packet resubmission is to increase the capacity and flexibility of the packet processing pipeline. For example, because the same packet is processed by the ingress pipeline multiple times, it effectively increase the amount of operations on the packet by N folds, where N is the number of times the packet is resubmitted.

Another use case is to deploy multiple packet processing algorithms on the same packet. For example, the original packet can be parsed and resubmitted in the first pass with additional metadata to select one of the algorithms. Then, the resubmitted packet can be parsed, modified and deparsed using the selected algorithm.

To facilitate communication from the ingress processing pass that caused a resubmit to occur, to the next ingress processing pass after the resubmit has happened, the resubmission mechanism supports attaching optional metadata with the resubmitted packet. The metadata is generated during the pass through the ingress pipeline that chooses the resubmit operation, and used in the next pass.

A PSA implementation provides a configuration bit `resubmit` to the PRE to enable the resubmission mechanism. If true, the original packet is resubmitted with the optional resubmit metadata. If false, the resubmission mechanism is disabled and no assignments to `resubmit_meta` should be performed.

7.11. Packet Recirculation

Packet recirculation is a mechanism to repeat ingress processing on a packet, after it has completed egress processing. Unlike a resubmit, where the resubmitted packet contents are identical to the packet that arrived at the ingress parser, a recirculated packet may have different headers than the packet had before recirculation. This could be useful in implementing features such as multiple levels of tunnel encapsulation or decapsulation.

Whether a packet is recirculated must be chosen during ingress processing, by sending the packet to port `PSA_PORT_RECIRCULATE`. Packet recirculation happens at the end of the egress pipeline. When a packet is sent to the recirculate port, the packet finishes egress processing, including the egress deparser, and then re-enters the ingress parser. The `ingress_port` of the recirculated packet is set to `PSA_PORT_RECIRCULATE`. The `packet_path` of the recirculated packet is set to `RECIRCULATE`.

Similar to packet resubmission, packet recirculation also supports attaching optional metadata with the recirculated packet. The metadata is generated during egress processing, and filled in by assigning a value to the `out` parameter `recirculate_meta` of the egress deparser. The metadata is available to the ingress parser after the packet is recirculated.

[1] https://en.wikipedia.org/wiki/Cut-through_switching

8. PSA Externs

8.1. Restrictions on where externs may be used

All instantiations in a $P4_{16}$ program occur at compile time, and can be arranged in a tree structure we will call the instantiation tree. The root of the tree **T** represents the top level of the program. Its children are the node for the package **PSA_Switch** described in [Chapter 6](#), and any externs instantiated at the top level of the program. The children of the **PSA_Switch** node are the packages and externs passed as parameters to the **PSA_Switch** instantiation. See [Figure 3](#) for a drawing of the smallest instantiation tree possible for a P4 program written for PSA.

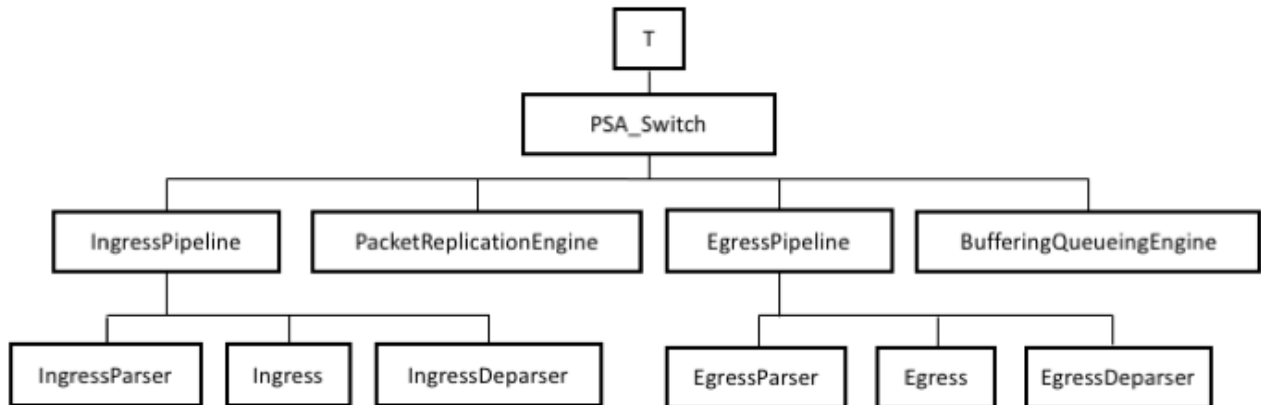


Figure 3. Minimal PSA instantiation tree

If any of those parsers or controls instantiate other parsers, controls, and/or externs, the instantiation tree contains child nodes for them, continuing until the instantiation tree is complete.

For every instance whose node is a descendant of the **Ingress** node in this tree, call it an **Ingress** instance. Similarly for the other ingress and egress parsers and controls. All other instances are top level instances.

A PSA implementation is allowed to reject programs that instantiate externs, or attempt to call their methods, from anywhere other than the places mentioned in Table {table-extern-usage}.

Table 5. Summary of controls that can instantiate and invoke externs.

Extern type	Where it may be instantiated and called from
ActionProfile	Ingress, Egress
ActionSelector	Ingress, Egress
Checksum	IngressParser, EgressParser, IngressDeparser, EgressDeparser
Counter	Ingress, Egress
Digest	IngressDeparser
DirectCounter	Ingress, Egress
DirectMeter	Ingress, Egress
Hash	Ingress, Egress
InternetChecksum	IngressParser, EgressParser, IngressDeparser, EgressDeparser

Extern type	Where it may be instantiated and called from
Meter	Ingress, Egress
Random	Ingress, Egress
Register	Ingress, Egress

For example, **Counter** being restricted to "Ingress, Egress" means that every **Counter** instance must be instantiated within either the **Ingress** control block or the **Egress** control block, or be a descendant of one of those nodes in the instantiation tree. If a **Counter** instance is instantiated in Ingress, for example, then it cannot be referenced, and thus its methods cannot be called, from any control block except **Ingress** or one of its descendants in the tree.

PSA implementations need not support instantiating these externs at the top level. PSA implementations are allowed to accept programs that use these externs in other places, but they need not. Thus P4 programmers wishing to maximize the portability of their programs should restrict their use of these externs to the places indicated in the table.

emit method calls for the type **packet_out** are restricted to be within deparser control blocks in PSA, because those are the only places where an instance of type **packet_out** is visible. Similarly all methods for type **packet_in**, e.g. **extract** and **advance**, are restricted to be within parsers in PSA programs. P4₁₆ restricts all **verify** method calls to be within parsers for all P4₁₆ programs, regardless of whether they are for the PSA.

Rationale:

- It is expected that the highest performance PSA implementations will not be able to update the same extern instance from both **Ingress** and **Egress**, nor from more than one of the parsers or controls defined in the PSA architecture.
- In a multi-pipeline device, there are effectively multiple instantiations of the ingress pipeline and of the egress pipeline. The primary motivation to create a multi-pipeline device is the practical difficulty in allowing the same stateful object (e.g. table, counter, etc.) to be accessed at a packet rate higher than that of a single pipeline. Thus each stateful object should be accessed from only a single pipeline on such a device. See [Appendix E](#).

8.2. PSA Table Properties

Table {table-table-properties} lists all P4 table properties defined by PSA that are not included in the base P4₁₆ language specification.

Table 6. Summary of PSA table properties

Property name	Type	See also
psa_direct_counter	one DirectCounter instance name	Section 8.7.3
psa_direct_meter	one DirectMeter instance name	Section 8.8
psa_implementation	instance name of one ActionProfile or ActionSelector	Section 8.11 , Section 8.12
psa_empty_group_action	action	Section 8.12
psa_idle_timeout	PSA_IdleTimeout_t	Section 8.2.1

A PSA implementation need not support both of a `psa_implementation` and `psa_direct_counter` property on the same table.

Similarly, a PSA implementation need not support both of a `psa_implementation` and `psa_direct_meter` property on the same table.

A PSA implementation must implement tables that have both a `psa_direct_counter` and `psa_direct_meter` property.

A PSA implementation need not support both `psa_implementation` and `psa_idle_timeout` properties on the same table.

8.2.1. Table entry timeout notification

PSA defines the table property `psa_idle_timeout` to enable specifying whether a table should maintain an idle time for each of its entries, and if so, what the data plane should do when a table entry has not been matched for a length of time at least its configured idle time.

The value assigned to `psa_idle_timeout` must be a value of type `PSA_IdleTimeout_t`:

```
/// Supported values for the psa_idle_timeout table property
enum PSA_IdleTimeout_t {
    NO_TIMEOUT,
    NOTIFY_CONTROL
};
```

If the property `psa_idle_timeout` is not specified for a table, its default value is `NO_TIMEOUT`. Such tables need not maintain an idle time for any of its table entries, and will not perform any special action regardless of how long a table entry remains unmatched.

If the property `psa_idle_timeout` is assigned a value of `NOTIFY_CONTROL` for a table `t`, this enables idle timeout notifications for the table. There is an API for the control plane to set idle timeout values for entries of table `t`.

If a table entry `e` has not been "hit" (ie; not matched by an invocation of `t.apply()`) for a time interval at least as long as its idle timeout value, the device should generate a notification to the control plane for entry `e`. The rate and mode of how the notifications are generated and delivered to the control plane are subject to configuration parameters specified by the control plane API.

Example:

```
table t {
    action a1 () { ... }
    action a2 () { ... }
    key = { hdr.f1: exact; }
    actions = { a1; a2; }
    default_action = a2;
    psa_idle_timeout = PSA_IdleTimeout_t.NOTIFY_CONTROL;
}
```

Restrictions on the idle timeout values and notifications:

- It is likely that any hardware implementation will have a limited number of bits to represent the values, and, since the values are programmed at runtime, it is the responsibility of the runtime

(P4Runtime or other controller software) to guarantee that the idle timeout values can be represented in the device. This can be done by scaling the values to the number of bits available on the platform, ensuring that the range of values between different entries are representable. A PSA implementation should only enable the programming of such tables, and return an error if the device does not support the idle timeout at all.

- If no value is programmed for a table entry, even though the table has enabled the idle timeout property, the entry will not generate a notification.
- PSA does not require a timeout value for a default action entry. The reason for not making this mandatory in the specification is that the default action may not have an explicit table entry to represent it, and also there are no known compelling use cases for a controller knowing when no misses have occurred for a particular table for a long time. The default action entry will not be aged out.
- Currently, tables implemented using `ActionSelectors` and `ActionProfiles` do not support the `psa_idle_timeout` property. Future versions of the specification may remove this restriction.

Devices are allowed to send idle timeout notifications on a best effort basis. For example, if a table has one million entries, and every entry reaches its idle timeout duration at the same instant in time, an implementation cannot send all of these notifications instantaneously, but only at some finite rate. Possible behaviors include:

- If the control plane deletes entry `e`, the deletion might occur even though the device never notified the control plane that `e`'s idle timeout was reached before it was deleted.
- If entry `e` is matched some time after its idle timeout has elapsed, and the device has not yet generated an idle timeout notification for `e`, it is possible that the device will never generate a notification to the control plane for that entry.

8.3. Packet Replication Engine

The `PacketReplicationEngine` extern (abbreviated PRE) represents a part of the PSA pipeline that is not programmable via writing P4 code.

Even though the PRE can not be programmed using P4, it can be configured using control plane APIs, e.g. configuring multicast groups and clone sessions. For every packet, your P4 program will typically assign values to intrinsic metadata in structs such as those of type `psa_ingress_output_metadata_t` and `psa_egress_output_metadata_t`, which direct the operation of the PRE on that packet. The file `psa.p4` defines some actions to help set these metadata fields for some common use cases, described in [Section 7.4](#) and [Section 7.7](#).

The PRE extern must be instantiated exactly once, in the `PSA_Switch` package instantiation. See near the end of [Chapter 6](#) for the package definitions from `psa.p4`. See below for an example of instantiating these packages, including the instantiation of one instance of `PacketReplicationEngine` and one of `BufferingQueueingEngine` in the `PSA_Switch` package instantiation.

```
IngressPipeline(IngressParserImpl(),
                ingress(),
                IngressDeparserImpl()) ip;

EgressPipeline(EgressParserImpl(),
               egress(),
               EgressDeparserImpl()) ep;

PSA_Switch(ip, PacketReplicationEngine(), ep, BufferingQueueingEngine()) main;
```

8.4. Buffering Queuing Engine

The `BufferingQueueingEngine` extern (abbreviated BQE) represents another part of the PSA pipeline, after egress, that is not programmable via writing P4 code.

Even though the BQE can not be programmed using P4, it can be configured both directly using control plane APIs and by setting intrinsic metadata.

The BQE extern must be instantiated exactly once, as the PRE must. See [Section 8.3](#) for additional discussion and example code.

8.5. Hashes

Supported hash algorithms:

```
enum PSA_HashAlgorithm_t {
    IDENTITY,
    CRC32,
    CRC32_CUSTOM,
    CRC16,
    CRC16_CUSTOM,
    ONES_COMPLEMENT16,    /// One's complement 16-bit sum used for IPv4 headers,
                          /// TCP, and UDP.
    TARGET_DEFAULT        /// target implementation defined
}
```

8.5.1. Hash function

Example usage:

```
parser P() {
    Hash<bit<16>>(PSA_HashAlgorithm_t.CRC16) h;
    bit<16> hash_value = h.get_hash(buffer);
}
```

Parameters:

- `algo`—The algorithm to use for computation (see [Section 8.5](#)).
- `O`—The type of the return value of the hash.

```
extern Hash<O> {
    /// Constructor
    Hash(PSA_HashAlgorithm_t algo);

    /// Compute the hash for data.
    /// @param data The data over which to calculate the hash.
    /// @return The hash value.
    @pure
    O get_hash<D>(in D data);

    /// Compute the hash for data, with modulo by max, then add base.
    /// @param base Minimum return value.
    /// @param data The data over which to calculate the hash.
    /// @param max The hash value is divided by max to get modulo.
}
```

```

///      An implementation may limit the largest value supported,
///      e.g. to a value like 32, or 256, and may also only
///      support powers of 2 for this value. P4 developers should
///      limit their choice to such values if they wish to
///      maximize portability.
/// @return (base + (h % max)) where h is the hash value.
@pure
0 get_hash<T, D>(in T base, in D data, in T max);
}

```

8.6. Checksums

PSA provides checksum functions compute an integer on the stream of bytes in packet headers. Checksums are often used as an integrity check to detect corrupted or otherwise malformed packets.

8.6.1. Basic checksum

The basic checksum extern provided in PSA supports arbitrary hash algorithms.

Parameters:

- **W**—The width of the checksum

```

extern Checksum<W> {
  /// Constructor
  Checksum(PSA_HashAlgorithm_t hash);

  /// Reset internal state and prepare unit for computation.
  /// Every instance of a Checksum object is automatically initialized as
  /// if clear() had been called on it. This initialization happens every
  /// time the object is instantiated, that is, whenever the parser or control
  /// containing the Checksum object are applied.
  /// All state maintained by the Checksum object is independent per packet.
  void clear();

  /// Add data to checksum
  void update<T>(in T data);

  /// Get checksum for data added (and not removed) since last clear
  @noSideEffects
  W get();
}

```

8.6.2. Incremental checksum

PSA also provides an incremental checksum that comes equipped with an additional **subtract** method that can be used to remove data previously added. The checksum is computed using the **ONES_COMPLEMENT16** hash algorithm used with protocols such as IPv4, TCP, and UDP—see [IETF RFC 1624](<https://tools.ietf.org/html/rfc1624>) and [Appendix B](#) for details.

```

// Checksum based on `ONES_COMPLEMENT16` algorithm used in IPv4, TCP, and UDP.
// Supports incremental updating via `subtract` method.
// See IETF RFC 1624.
extern InternetChecksum {
  /// Constructor
  InternetChecksum();
}

```

```

/// Reset internal state and prepare unit for computation. Every
/// instance of an InternetChecksum object is automatically
/// initialized as if clear() had been called on it, once for each
/// time the parser or control it is instantiated within is
/// executed. All state maintained by it is independent per packet.
void clear();

/// Add data to checksum. data must be a multiple of 16 bits long.
void add<T>(in T data);

/// Subtract data from existing checksum. data must be a multiple of
/// 16 bits long.
void subtract<T>(in T data);

/// Get checksum for data added (and not removed) since last clear
@noSideEffects
bit<16> get();

/// Get current state of checksum computation. The return value is
/// only intended to be used for a future call to the set_state
/// method.
@noSideEffects
bit<16> get_state();

/// Restore the state of the InternetChecksum instance to one
/// returned from an earlier call to the get_state method. This
/// state could have been returned from the same instance of the
/// InternetChecksum extern, or a different one.
void set_state(in bit<16> checksum_state);
}

```

8.6.3. InternetChecksum examples

The partial program below demonstrates one way to use the `InternetChecksum` extern to verify whether the checksum field in a parsed IPv4 header is correct, and set a parser error if it is wrong. It also demonstrates checking for parser errors in the `Ingress` control block, dropping the packet if any errors occurred during parsing. PSA programs may choose to handle packets with parser errors in other ways than shown in this example—it is up to the P4 program author to choose and write the desired behavior.

Neither `P416` nor the PSA provide any special mechanisms to record the location within a packet that a parser error occurred. A P4 program author can choose to record such location information explicitly. For example, one may define metadata fields specifically for that purpose—e.g. to hold an encoded value representing the last parser metadata reached, or the number of bytes extracted so far—and then assign values to those fields within the parser state code.

```

// Define additional error values, one of them for packets with
// incorrect IPv4 header checksums.
error {
    UnhandledIPv4Options,
    BadIPv4HeaderChecksum
}

typedef bit<32> PacketCounter_t;
typedef bit<8> ErrorIndex_t;

const bit<9> NUM_ERRORS = 256;

parser IngressParserImpl(packet_in buffer,

```

```

        out headers hdr,
        inout metadata user_meta,
        in psa_ingress_parser_input_metadata_t istd,
        in empty_metadata_t resubmit_meta,
        in empty_metadata_t recirculate_meta)
{
    InternetChecksum() ck;
    state start {
        buffer.extract(hdr.etherType);
        transition select(hdr.etherType) {
            0x0800: parse_ipv4;
            default: accept;
        }
    }
    state parse_ipv4 {
        buffer.extract(hdr.ipv4);
        // TBD: It would be good to enhance this example to
        // demonstrate checking of IPv4 header checksums for IPv4
        // headers with options, but this example does not handle such
        // packets.
        verify(hdr.ipv4.ihl == 5, error.UnhandledIPv4Options);
        ck.clear();
        ck.add({
            /* 16-bit word 0 */ hdr.ipv4.version, hdr.ipv4.ihl, hdr.ipv4.diffserv,
            /* 16-bit word 1 */ hdr.ipv4.totalLen,
            /* 16-bit word 2 */ hdr.ipv4.identification,
            /* 16-bit word 3 */ hdr.ipv4.flags, hdr.ipv4.fragOffset,
            /* 16-bit word 4 */ hdr.ipv4.ttl, hdr.ipv4.protocol,
            /* 16-bit word 5 skip hdr.ipv4.hdrChecksum, */
            /* 16-bit words 6-7 */ hdr.ipv4.srcAddr,
            /* 16-bit words 8-9 */ hdr.ipv4.dstAddr
        });
        // The verify statement below will cause the parser to enter
        // the reject state, and thus terminate parsing immediately,
        // if the IPv4 header checksum is wrong. It will also record
        // the error error.BadIPv4HeaderChecksum, which will be
        // available in a metadata field in the ingress control block.
        verify(ck.get() == hdr.ipv4.hdrChecksum,
            error.BadIPv4HeaderChecksum);
        transition select(hdr.ipv4.protocol) {
            6: parse_tcp;
            default: accept;
        }
    }
    state parse_tcp {
        buffer.extract(hdr.tcp);
        transition accept;
    }
}

control ingress(inout headers hdr,
    inout metadata user_meta,
    in psa_ingress_input_metadata_t istd,
    inout psa_ingress_output_metadata_t ostd)
{
    // Table parser_error_count_and_convert below shows one way to
    // count the number of times each parser error was encountered.
    // Although it is not used in this example program, it also shows
    // how to convert the error value into a unique bit vector value
    // 'error_idx', which can be useful if you wish to put a bit
    // vector encoding of an error into a packet header, e.g. for a
    // packet sent to the control CPU.

```



```

DirectCounter<PacketCounter_t>(PSA_CounterType_t.PACKETS) parser_error_counts;
ErrorIndex_t error_idx;

action set_error_idx (ErrorIndex_t idx) {
    error_idx = idx;
    parser_error_counts.count();
}
table parser_error_count_and_convert {
    key = {
        istd.parser_error : exact;
    }
    actions = {
        set_error_idx;
    }
    default_action = set_error_idx(0);
    const entries = {
        error.NoError           : set_error_idx(1);
        error.PacketTooShort    : set_error_idx(2);
        error.NoMatch           : set_error_idx(3);
        error.StackOutOfBounds   : set_error_idx(4);
        error.HeaderTooShort     : set_error_idx(5);
        error.ParserTimeout      : set_error_idx(6);
        error.BadIPv4HeaderChecksum : set_error_idx(7);
        error.UnhandledIPv4Options : set_error_idx(8);
    }
    psa_direct_counter = parser_error_counts;
}
}
apply {
    if (istd.parser_error != error.NoError) {
        // Example code showing how to count number of times each
        // kind of parser error was seen.
        parser_error_count_and_convert.apply();
        ingress_drop(ostd);
        exit;
    }
    // Do normal packet processing here.
}
}

```

The partial program below demonstrates one way to use the `InternetChecksum` extern to calculate and then fill in a correct IPv4 header checksum in the deparser block. In this example, the checksum is calculated fresh, so the outgoing checksum will be correct regardless of what changes might have been made to the IPv4 header fields in the Ingress (or Egress) control block that precedes it.

```

control EgressDeparserImpl(packet_out packet,
                           out empty_metadata_t clone_e2e_meta,
                           out empty_metadata_t recirculate_meta,
                           inout headers hdr,
                           in metadata meta,
                           in psa_egress_output_metadata_t istd,
                           in psa_egress_deparser_input_metadata_t edstd)
{
    InternetChecksum() ck;
    apply {
        ck.clear();
        ck.add({
            /* 16-bit word 0 */ /* hdr.ipv4.version, hdr.ipv4.ihl, hdr.ipv4.diffserv,
            /* 16-bit word 1 */ /* hdr.ipv4.totalLen,
            /* 16-bit word 2 */ /* hdr.ipv4.identification,
            /* 16-bit word 3 */ /* hdr.ipv4.flags, hdr.ipv4.fragOffset,
            /* 16-bit word 4 */ /* hdr.ipv4.ttl, hdr.ipv4.protocol,

```

```

        /* 16-bit word 5 skip hdr.ipv4.hdrChecksum, */
        /* 16-bit words 6-7 */ hdr.ipv4.srcAddr,
        /* 16-bit words 8-9 */ hdr.ipv4.dstAddr
    });
    hdr.ipv4.hdrChecksum = ck.get();
    packet.emit(hdr.ethernet);
    packet.emit(hdr.ipv4);
    packet.emit(hdr.tcp);
}
}

```

As a final example, we can use the `InternetChecksum` to compute an incremental checksum for the TCP header. Recall the TCP checksum is computed over the *entire* packet, including the payload. Because the packet payload need not be available in a PSA implementation, we assume that the TCP checksum on the original packet is correct, and update it incrementally by invoking `subtract` and then `add` on any fields that are modified by the program. For example, the `Ingress` control in the program below updates the IPv4 source address, recording the original source address in a metadata field:

```

control ingress(inout headers hdr,
               inout metadata user_meta,
               in  psa_ingress_input_metadata_t istd,
               inout psa_ingress_output_metadata_t ostd) {
    action drop() {
        ingress_drop(ostd);
    }
    action forward(PortId_t port, bit<32> srcAddr) {
        user_meta.fwd_metadata.old_srcAddr = hdr.ipv4.srcAddr;
        hdr.ipv4.srcAddr = srcAddr;
        send_to_port(ostd, port);
    }
    table route {
        key = { hdr.ipv4.dstAddr : lpm; }
        actions = {
            forward;
            drop;
        }
    }
    apply {
        if(hdr.ipv4.isValid()) {
            route.apply();
        }
    }
}

```

The deparser first updates the IPv4 checksum as above, and then incrementally computes the TCP checksum.

```

control EgressDeparserImpl(packet_out packet,
                          out empty_metadata_t clone_e2e_meta,
                          out empty_metadata_t recirculate_meta,
                          inout headers hdr,
                          in metadata user_meta,
                          in psa_egress_output_metadata_t istd,
                          in psa_egress_deparser_input_metadata_t edstd)
{
    InternetChecksum() ck;
    apply {
        // Update IPv4 checksum
        // This clear() call can be removed without affecting
    }
}

```

```

// behavior, as an InternetChecksum instance is automatically
// cleared for each packet.
ck.clear();
ck.add({
    /* 16-bit word 0 */ hdr.ipv4.version, hdr.ipv4.ihl, hdr.ipv4.diffserv,
    /* 16-bit word 1 */ hdr.ipv4.totalLen,
    /* 16-bit word 2 */ hdr.ipv4.identification,
    /* 16-bit word 3 */ hdr.ipv4.flags, hdr.ipv4.fragOffset,
    /* 16-bit word 4 */ hdr.ipv4.ttl, hdr.ipv4.protocol,
    /* 16-bit word 5 skip hdr.ipv4.hdrChecksum, */
    /* 16-bit words 6-7 */ hdr.ipv4.srcAddr,
    /* 16-bit words 8-9 */ hdr.ipv4.dstAddr
});
hdr.ipv4.hdrChecksum = ck.get();
// Update TCP checksum
// This clear() call is necessary for correct behavior, since
// the same instance 'ck' is reused from above for the same
// packet. If a second InternetChecksum instance other than
// 'ck' were used below instead, this clear() call would be
// unnecessary.
ck.clear();
// Subtract the original TCP checksum
ck.subtract(hdr.tcp.checksum);
// Subtract the effect of the original IPv4 source address,
// which is part of the TCP 'pseudo-header' for the purposes
// of TCP checksum calculation (see RFC 793), then add the
// effect of the new IPv4 source address.
ck.subtract(user_meta.fwd_metadata.old_srcAddr);
ck.add(hdr.ipv4.srcAddr);
hdr.tcp.checksum = ck.get();
packet.emit(hdr.ethernet);
packet.emit(hdr.ipv4);
packet.emit(hdr.tcp);
}
}

```

8.7. Counters

Counters are a mechanism for keeping statistics. The control plane can read counter values. A P4 program cannot read counter values, only update them. If you wish to implement a feature involving sequence numbers in packets, for example, use Registers instead ([Section 8.9](#)).

Direct counters are counters associated with a particular P4 table, and are implemented by the extern **DirectCounter**. There are also indexed counters, which are implemented by the extern **Counter**. The primary differences between direct counters and indexed counters are:

- Number of independently updatable counter values:
 - A single instantiation of a direct counter always contains as many independent counter values as the number of entries in the table with which it is associated.
 - You must specify the number of independent counter values for an indexed counter when instantiating it. This number of counters need not be the same as the size of any table.
- Where counter updates are allowed in the P4 program:
 - For a direct counter, you may only invoke its **count** method from inside the actions of the table with which it is associated, and this always updates the counter value associated with the matching table entry.
 - For an indexed counter, you may invoke its **count** method anywhere in the P4 program where

extern object method invocations are permitted (e.g. inside actions, or directly inside a control's **apply** block), and every such invocation must specify the index of the counter value to be updated.

Counters are only intended to support packet counters and byte counters, or a combination of both called **PACKETS_AND_BYTES**. The byte counts are always increased by some measure of the packet length, where the packet length used might vary from one PSA implementation to another. For example, one implementation might use the Ethernet frame length, including the Ethernet header and FCS bytes, as the packet arrived on a physical port. Another might not include the FCS bytes in its definition of the packet length. Another might only include the Ethernet payload length. Each PSA implementation should document how it determines the packet length used for byte counter updates.

If you wish to keep counts of other quantities, or to have more precise control over the packet length used in a byte counter, you may use Registers to achieve that ([Section 8.9](#)).

8.7.1. Counter types

```
enum PSA_CounterType_t {
    PACKETS,
    BYTES,
    PACKETS_AND_BYTES
}
```

8.7.2. Counter

```
/// Indirect counter with n_counters independent counter values, where
/// every counter value has a data plane size specified by type W.

@noWarn("unused")
extern Counter<W, S> {
    Counter(bit<32> n_counters, PSA_CounterType_t type);
    void count(in S index);
}
```

See [Appendix C](#) for pseudocode of an example implementation of the Counter extern.

PSA implementations must not update any counter values if an indexed counter is updated with an index that is too large. It is recommended that they count such erroneous attempted updates, and record other information that can help an P4 programmer debug such errors.

8.7.3. Direct Counter

```
@noWarn("unused")
extern DirectCounter<W> {
    DirectCounter(PSA_CounterType_t type);
    void count();
}
```

A **DirectCounter** instance must appear as the value of the **psa_direct_counter** table attribute for at most one table. We call this table the **DirectCounter** instance's "owner". It is an error to call the **count** method for a **DirectCounter** instance anywhere except inside an action of its owner table.

The counter value updated by an invocation of **count** is always the one associated with the table entry

that matched.

An action of an owner table need not have `count` method calls for all of the `DirectCounter` instances that the table owns. You must use an explicit `count()` method call on a `DirectCounter` to update it, otherwise its state will not change.

An example implementation for the `DirectCounter` extern is essentially the same as the one for `Counter`. Since there is no `index` parameter to the `count` method, there is no need to check for whether it is in range.

The rules here mean that an action that calls `count` on a `DirectCounter` instance may only be an action of that instance's one owner table. If you want to have a single action `A` that can be invoked by multiple tables, you can still do so by having a unique action for each such table with a `DirectCounter`, where each such action in turn calls action `A`, in addition to any `count` invocations they have.

A `DirectCounter` instance must have a counter value associated with its owner table that is updated when there is a default action assigned to the table, and a search of the table results in a miss. If there is no default action assigned to the table, then there need not be any counter updated when a search of the table results in a miss.

By "a default action is assigned to a table", we mean that either the table has a `default_action` table property with an action assigned to it in the P4 program, or the control plane has made an explicit call to assign the table a default action. If neither of these is true, then there is no default action assigned to the table.

8.7.4. Example program using counters

The following partial P4 program demonstrates the instantiation and updating of `Counter` and `DirectCounter` externs.

```
typedef bit<48> ByteCounter_t;
typedef bit<32> PacketCounter_t;
typedef bit<80> PacketByteCounter_t;

const bit<32> NUM_PORTS = 512;

struct headers {
    ethernet_t    ethernet;
    ipv4_t        ipv4;
}

control ingress(inout headers hdr,
               inout metadata user_meta,
               in   psa_ingress_input_metadata_t istd,
               inout psa_ingress_output_metadata_t ostd)
{
    Counter<ByteCounter_t, PortId_t>(NUM_PORTS, PSA_CounterType_t.BYTES)
        port_bytes_in;
    DirectCounter<PacketByteCounter_t>(PSA_CounterType_t.PACKETS_AND_BYTES)
        per_prefix_pkt_byte_count;

    action next_hop(PortId_t oport) {
        per_prefix_pkt_byte_count.count();
        send_to_port(ostd, oport);
    }
    action default_route_drop() {
        per_prefix_pkt_byte_count.count();
    }
}
```

```

        ingress_drop(ostd);
    }
    table ipv4_da_lpm {
        key = { hdr.ipv4.dstAddr: lpm; }
        actions = {
            next_hop;
            default_route_drop;
        }
        default_action = default_route_drop;
        // table ipv4_da_lpm owns this DirectCounter instance
        psa_direct_counter = per_prefix_pkt_byte_count;
    }
    apply {
        port_bytes_in.count(istd.ingress_port);
        if (hdr.ipv4.isValid()) {
            ipv4_da_lpm.apply();
        }
    }
}

control egress(inout headers hdr,
               inout metadata user_meta,
               in    psa_egress_input_metadata_t istd,
               inout psa_egress_output_metadata_t ostd)
{
    Counter<ByteCounter_t, PortId_t>(NUM_PORTS, PSA_CounterType_t.BYTES)
    port_bytes_out;
    apply {
        // By doing these stats updates on egress, then because
        // multicast replication happens before egress processing,
        // this update will occur once for each copy made, which in
        // this example is intentional.
        port_bytes_out.count(istd.egress_port);
    }
}

```

8.8. Meters

Meters (RFC 2698) are a more complex mechanism for keeping statistics about packets, most often used for dropping or "marking" packets that exceed an average packet or bit rate. To mark a packet means to change one or more of its quality of service values in packet headers such as the 802.1Q PCP (priority code point) or DSCP (differentiated service code point) bits within the IPv4 or IPv6 type of service byte. The meters specified in the PSA are 3-color meters.

PSA meters do not require any particular drop or marking actions, nor do they automatically implement those behaviors for you. Meters keep enough state, and update their state during `execute()` method calls, in such a way that they return a **GREEN** (also known as conform), **YELLOW** (exceed), or **RED** (violate) result. See RFC 2698 for details on the conditions under which one of these three results is returned. The P4 program is responsible for examining that returned result, and making changes to packet forwarding behavior as a result. The value returned by an uninitialized meter shall be GREEN. This is in accordance with the P4Runtime specification.

RFC 2698 describes "color aware" and "color blind" variations of meters. The **Meter** and **DirectMeter** externs implement both. The only difference is in which `execute` method you use when updating them. See the comments on the `extern` definitions below.

Similar to counters, there are two flavors of meters: indexed and direct. (Indexed) meters are addressed by index, while direct meters always update a meter state corresponding to the matched table entry or

action, and from the control plane API are addressed using P4Runtime table entry as key.

There are many other similarities between counters and meters, including:

- The number of independently updatable meter values.
- Where meter updates are allowed in a P4 program.
- For **BYTES** type meters, the packet length used in the update is determined by the PSA implementation, and can vary from one PSA implementation to another.

Further similarities between direct counters and direct meters include:

- **DirectMeter execute** method calls must be performed within actions invoked by the table that owns the **DirectMeter** instance. It is optional for such an action to call the **execute** method.
- There must be a meter state associated with a **DirectMeter** instance's owner table, that can be updated when the table result is a miss. As for a **DirectCounter**, this state only needs to exist if a default action is assigned to the table.

The table attribute to specify that a table owns a **DirectMeter** instance is **psa_direct_meter**. The value of this table attribute is a **DirectMeter** instance name.

As for counters, if you call the **execute(idx)** method on an indexed meter and **idx** is at least the number of meter states, so **idx** is out of range, no meter state is updated. The **execute** call still returns a value of type **PSA_MeterColor_t**, but the value is undefined—programs that wish to have predictable behavior across implementations must not use the undefined value in a way that affects the output packet or other side effects. The example code below shows one way to achieve predictable behavior. Note that this undefined behavior cannot occur if the value of **n_meters** of an indexed meter is 2^W , and the type **S** used to construct the meter is **bit<W>**, since the index value could never be out of range.

```
#define METER1_SIZE 100
Meter<bit<7>>(METER1_SIZE, PSA_MeterType_t.BYTES) meter1;
bit<7> idx;
PSA_MeterColor_t color1;

// ... later ...

if (idx < METER1_SIZE) {
    color1 = meter1.execute(idx, PSA_MeterColor_t.GREEN);
} else {
    // If idx is out of range, use a default value for color1. One
    // may also choose to store an error flag in some metadata field.
    color1 = PSA_MeterColor_t.RED;
}
```

Any implementation will have a finite range that can be specified for the Peak Burst Size and Committed Burst Size. An implementation should document the maximum burst sizes they support, and if the implementation internally truncates the values that the control plane requests to something more coarse than any number of bytes, that should also be documented. It is recommended that the maximum burst sizes be allowed as large as the number of bytes that can be transmitted across the implementation's maximum speed port in 100 milliseconds.

Implementations will also have finite ranges and precisions that they support for the Peak Information Rate and Committed Information Rate. An implementation should document the maximum rate it supports, as well as the precision it supports for implementing requested rates. It is recommended that the maximum rate supported be at least the rate of the implementation's fastest port, and that the actual

implemented rate should always be within plus or minus 0.1% of the requested rate.

8.8.1. Meter types

```
enum PSA_MeterType_t {  
    PACKETS,  
    BYTES  
}
```

8.8.2. Meter colors

```
enum PSA_MeterColor_t { RED, GREEN, YELLOW }
```

8.8.3. Meter

```
// Indexed meter with n_meters independent meter states.  
  
extern Meter<S> {  
    Meter(bit<32> n_meters, PSA_MeterType_t type);  
  
    // Use this method call to perform a color aware meter update (see  
    // RFC 2698). The color of the packet before the method call was  
    // made is specified by the color parameter.  
    PSA_MeterColor_t execute(in S index, in PSA_MeterColor_t color);  
  
    // Use this method call to perform a color blind meter update (see  
    // RFC 2698). It may be implemented via a call to execute(index,  
    // MeterColor_t.GREEN), which has the same behavior.  
    PSA_MeterColor_t execute(in S index);  
}
```

8.8.4. Direct Meter

```
extern DirectMeter {  
    DirectMeter(PSA_MeterType_t type);  
    // See the corresponding methods for extern Meter.  
    PSA_MeterColor_t execute(in PSA_MeterColor_t color);  
    PSA_MeterColor_t execute();  
}
```

8.9. Registers

Registers are stateful memories whose values can be read and written during packet forwarding under the control of the P4 program. They are similar to counters and meters in that their state can be modified as a result of processing packets, but they are far more general in the behavior they can implement.

Although you may not use register contents directly in table match keys, you may use the `read()` method call on the right-hand side of an assignment statement, which retrieves the current value of the register. You may copy the register value into metadata, and it is then available for matching in subsequent tables.

There are two different constructors for Register instances. The value returned for the uninitialized

variant is undefined. The value returned for the initialized variant is the one specified by the `initial_value` parameter of the constructor.

A simple usage example is to verify that a "first packet" was seen for a particular type of flow. A register cell would be allocated to the flow, initialized to "clear". When the protocol signaled a "first packet", the table would match on this value and update the flow's cell to "marked". Subsequent packets in the flow would be mapped to the same cell; the current cell value would be stored in metadata for the packet and a subsequent table could check that the flow was marked as active.

```
extern Register<T, S> {
    /// Instantiate an array of <size> registers. The initial value is
    /// undefined.
    Register(bit<32> size);
    /// Initialize an array of <size> registers and set their value to
    /// initial_value.
    Register(bit<32> size, T initial_value);

    @noSideEffects
    T read (in S index);
    void write (in S index, in T value);
}
```

Another example using registers is given below. It implements a packet and byte counter, where the byte counter can be updated by a packet length specified in the P4 program, rather than one chosen by the PSA implementation.

```
const bit<32> NUM_PORTS = 512;

// It would be more convenient to use a struct type to represent the
// state of a combined packet and byte count, and many other compound
// values one might wish to store in a Register instance. However,
// the latest p4test as of 2018-Feb-10 does not allow a struct type to
// be returned from a method call like Register.read().

// Refer to this Github issue for status of generalizing this:
// https://github.com/p4lang/p4-spec/issues/383

#define PACKET_COUNT_WIDTH 32
#define BYTE_COUNT_WIDTH 48
// #define PACKET_BYTE_COUNT_WIDTH (PACKET_COUNT_WIDTH + BYTE_COUNT_WIDTH)
#define PACKET_BYTE_COUNT_WIDTH 80

#define PACKET_COUNT_RANGE (PACKET_BYTE_COUNT_WIDTH-1):BYTE_COUNT_WIDTH
#define BYTE_COUNT_RANGE (BYTE_COUNT_WIDTH-1):0

typedef bit<PACKET_BYTE_COUNT_WIDTH> PacketByteCountState_t;

action update_pkt_ip_byte_count (inout PacketByteCountState_t s,
                                in bit<16> ip_length_bytes)
{
    s[PACKET_COUNT_RANGE] = s[PACKET_COUNT_RANGE] + 1;
    s[BYTE_COUNT_RANGE] = (s[BYTE_COUNT_RANGE] +
                           (bit<BYTE_COUNT_WIDTH>) ip_length_bytes);
}

control ingress(inout headers hdr,
                inout metadata user_meta,
                in psa_ingress_input_metadata_t istd,
                inout psa_ingress_output_metadata_t ostd)
{
```

```

Register<PacketByteCountState_t, PortId_t>(NUM_PORTS)
    port_pkt_ip_bytes_in;

    apply {
        ostd.egress_port = (PortId_t) 0;
        if (hdr.ipv4.isValid()) {
            @atomic {
                PacketByteCountState_t tmp;
                tmp = port_pkt_ip_bytes_in.read(istd.ingress_port);
                update_pkt_ip_byte_count(tmp, hdr.ipv4.totalLen);
                port_pkt_ip_bytes_in.write(istd.ingress_port, tmp);
            }
        }
    }
}

```

Note the use of the `@atomic` annotation in the block enclosing the `read()` and `write()` method calls on the `Register` instance. It is expected to be common that register accesses will need the `@atomic` annotation around portions of your program in order to behave as you desire. As stated in the P4₁₆ specification, without the `@atomic` annotation in this example, an implementation is allowed to process two packets `P1` and `P2` in parallel, and perform the register access operations in this order:

```

// Possible order of operations for the example program if the
// @atomic annotation is _not_ used.

tmp = port_pkt_ip_bytes_in.read(istd.ingress_port); // for packet P1
tmp = port_pkt_ip_bytes_in.read(istd.ingress_port); // for packet P2

// At this time, if P1 and P2 came from the same ingress_port,
// each of their values of tmp are identical.

update_pkt_ip_byte_count(tmp, hdr.ipv4.totalLen); // for packet P1
update_pkt_ip_byte_count(tmp, hdr.ipv4.totalLen); // for packet P2

port_pkt_ip_bytes_in.write(istd.ingress_port, tmp); // for packet P1
port_pkt_ip_bytes_in.write(istd.ingress_port, tmp); // for packet P2
// The write() from packet P1 is lost.

```

Since different implementations may have different upper limits on the complexity of code that they will accept within an `@atomic` block, we recommend you keep them as small as possible, subject to maintaining your desired correct behavior.

Individual counter and meter method calls need not be enclosed in `@atomic` blocks to be safe—they guarantee atomic behavior of their individual method calls, without losing any updates. Even though the P4₁₆ v1.0.0 language specification currently requires that every `action` of a table behave as if its entire body is annotated by an `@atomic` annotation, it is recommended to explicitly use `@atomic` annotations inside of action bodies as if this were not the case, since (a) it is harmless, and more importantly (b) this requirement may be removed in a near future revision of the language specification.

As for indexed counters and meters, access to an index of a register that is at least the size of the register is out of bounds. An out of bounds write has no effect on the state of the system. An out of bounds read returns an undefined value. See the example in [Section 8.8](#) for one way to write code to guarantee avoiding this undefined behavior. Out of bounds register accesses are impossible for a register instance with type `S` declared as `bit<W>` and size `$2^W$` entries.

8.10. Random

The **Random** extern provides generation of pseudo-random numbers in a specified range with a uniform distribution. If one wishes to generate numbers with a non-uniform distribution, you may do so by first generating a uniformly distributed random value, and then using appropriate table lookups and/or arithmetic on the resulting value to achieve the desired distribution.

An implementation is not required to produce cryptographically strong pseudo-random number generation. For example, a particularly inexpensive implementation might use a linear feedback shift register to generate values.

```
extern Random<T> {  
  
    /// Return a random value in the range [min, max], inclusive.  
    /// Implementations are allowed to support only ranges where (max -  
    /// min + 1) is a power of 2. P4 developers should limit their  
    /// arguments to such values if they wish to maximize portability.  
  
    Random(T min, T max);  
    T read();  
}
```

8.11. Action Profile

Action profiles are used as table implementation attributes.

Action profiles provide a mechanism to populate table entries with action specifications that have been defined outside the table entry specification. An action profile extern can be instantiated as a resource in the P4 program. A table that uses this action profile must specify its **psa_implementation** attribute as the action profile instance.

Table entry	Key (h.f. lpm)	Action spec.
t1	01001*	set_port(1)
t2	1100*	set_port(2)
t3	101*	set_port(1)

(a) Direct table.

Table entry	Key (h.f. lpm)	Member ref.
t1	01001*	m1
t2	1100*	m2
t3	101*	m1

Member ref.	Action spec.
m1	set_port(1)
m2	set_port(2)

(b) Indirect table with action profile implementation.

Figure 4. Action profiles in PSA

Figure 4 contrasts a direct table with a table that has an action profile implementation. A direct table, as seen in Figure 4 (a) contains the action specification in each table entry. In this example, the table has a

match key consisting of an LPM on header field `h.f`. The action is to set the port. As we can see, entries `t1` and `t3` have the same action, i.e. to set the port to 1. Action profiles enable sharing an action across multiple entries by using a separate table as shown in Figure 4 (b).

A table with an action profile implementation has entries that point to a member reference instead of directly defining an action specification. A mapping from member references to action specifications is maintained in a separate table that is part of the action profile instance defined in the table `psa_implementation` attribute. When a table with an action profile implementation is applied, the member reference is resolved and the corresponding action specification is applied to the packet.

Action profile members may only specify action types defined in the `actions` attribute of the implemented table. An action profile instance may be shared across multiple tables only if all such tables define the same set of actions in their `actions` attribute. Tables with an action profile implementation cannot define a default action. The default action for such tables is implicitly set to `NoAction`.

The control plane can add, modify or delete member entries for a given action profile instance. The controller-assigned member reference must be unique in the scope of the action profile instance. An action profile instance may hold at most `size` entries as defined in the constructor parameter. Table entries must specify the action using the controller-assigned reference for the desired member entry. Directly specifying the action as part of the table entry is not allowed for tables with an action profile implementation.

```
extern ActionProfile {  
    /// Construct an action profile of 'size' entries  
    ActionProfile(bit<32> size);  
}
```

8.11.1. Action Profile Example

The P4 control block `Ctrl` in the example below instantiates an action profile `ap` that can contain at most 128 member entries. Table `indirect` uses this instance by specifying the `psa_implementation` attribute. The control plane can add member entries to `ap`, where each member can specify either a `foo` or `NoAction` action. Table entries for `indirect` table must specify the action using the controller-assigned member reference.

```
control Ctrl(inout H hdr, inout M meta) {  
  
    action foo() { meta.foo = 1; }  
  
    ActionProfile(128) ap;  
  
    table indirect {  
        key = {hdr.ipv4.dst_address: exact;}  
        actions = { foo; NoAction; }  
        psa_implementation = ap;  
    }  
  
    apply {  
        indirect.apply();  
    }  
}
```

8.12. Action Selector

Action selectors are used as table implementation attributes.

Action selectors implement yet another mechanism to populate table entries with action specifications that have been defined outside the table entry. They are more powerful than action profiles because they also provide the ability to dynamically select the action specification to apply upon matching a table entry. An action selector extern can be instantiated as a resource in the P4 program, similar to action profiles. Furthermore, a table that uses this action selector must specify its `psa_implementation` attribute as the action selector instance.

Table entry	Key (h.f. lpm)	Member/ Group ref.	Group ref.	Members	Member ref.	Action spec.
t1	01001*	g1	g1	m1, m2	m1	set_port(1)
t2	1100*	m2	g2	m1	m2	set_port(2)
t3	101*	g2	g3	m2		

Figure 5. Action selectors in PSA

Figure 5 illustrates a table that has an action selector implementation. In this example, the table has a match key consisting of an LPM on header field `h.f`. A second match type `selector` is used to define the fields that are used to look up the action specification from the selector at runtime.

A table with an action action selector implementation consists of entries that point to either an action profile member reference or an action profile group reference. An action selector instance can be logically visualized as two tables as shown in Figure 5. The first table contains a mapping from group references to a set of member references. The second table contains a mapping from member references to action specifications.

When a packet matches a table entry at runtime, the controller-assigned reference of the action profile member or group is read. If the entry points to a member then the corresponding action specification is applied to the packet. However, if the entry points to a group, a dynamic selection algorithm is used to select a member from the group, and the action specification corresponding to that member is applied. The dynamic selection algorithm is specified as a parameter when instantiating the action selector.

8.12.1. Group Actions

This feature is currently supported only for P4Runtime^[2 - Chapter 4] one-shot action selector programming.

A group may be configured with a *group action* that executes logically before the selected member action. An action used as a group action must be annotated with `@groupaction`.

As specified in the P4Runtime specification, attempting to use an action annotated with `@groupaction` as a table entry action or as a table default action must fail and return an appropriate error status.

An action that is not annotated with `@groupaction` must not be used as a group action.

The `@groupaction` annotation is not allowed in combination with `@tableonly` or `@defaultonly`. (For the semantics of `@tableonly` and `@defaultonly`, see the P4 language specification.)

Because an action cannot be used both as a group action and as a table entry or default action, a P4 program that requires equivalent behavior in multiple contexts must define separate actions for each context.

Action selector members may only specify action types defined in the `actions` attribute of the implemented table.

Minimum requirements for a PSA implementation of action selectors:

- Support non-empty groups where every action in the same group has the same action name.
- Within the same group, support arbitrary action parameter values among different members of the group.
- Support different action names in different groups.
- No predictable data plane behavior is required if a table entry is matched that points at an empty group.

Optional extensions:

- Support non-empty groups where in the same group, different actions can have different action names, as well as arbitrary action parameter values.
- Support table entries that point at an empty group. When the entry is matched, execute the action assigned to the table property `psa_empty_group_action`.

The `psa_empty_group_action` property of a table is similar to the `default_action` property in the following ways:

- They both have actions as their values.
- The P4 source code specifies the initial value.
- If the table property `psa_empty_group_action` is not given in the P4 source code, its value is `NoAction()`.
- They may have a `const` modifier, indicating that control software is not allowed to change this action.
- In the absence of a `const` modifier, the control software is allowed to change the action assigned to `psa_empty_group_action`.

PSA implementers should note that supporting empty groups with predictable data plane behavior may be required in a future version of PSA. In some cases, it may be possible for the combination of a PSA data plane plus its P4Runtime server software to achieve this desired behavior, as far as the P4Runtime client controller software can observe. See [Appendix G](#).

An action selector instance may be shared across multiple tables only if all such tables define the same set of actions in their `actions` attribute. Furthermore, the selector match fields for such tables must be identical and must be specified in the same order across all tables sharing the selector. Tables with an action selector implementation cannot define a default action. The default action for such tables is implicitly set to `NoAction`.

The dynamic selection algorithm requires a field list as an input for generating the index to a member entry in a group. This field list is created by using the match type `selector` when defining the table match key. The match fields of type `selector` are composed into a field list in the order they are specified. The composed field list is passed as an input to the action selector implementation. It is illegal to define a `selector` type match field if the table does not have an action selector implementation.

The control plane can add, modify or delete member and group entries for a given action selector instance. An action selector instance may hold at most `size` member entries as defined in the constructor parameter. The number of groups may be at most the size of the table that is implemented by the selector. Table entries must specify the action using a reference to the desired member or group entry. Directly specifying the action as part of the table entry is not allowed for tables with an action selector

implementation.

```
extern ActionSelector {  
    /// Construct an action selector of 'size' entries  
    /// @param algo hash algorithm to select a member in a group  
    /// @param size number of entries in the action selector  
    /// @param outputWidth size of the key  
    ActionSelector(PSA_HashAlgorithm_t algo, bit<32> size, bit<32> outputWidth);  
}
```

8.12.2. Action Selector Example

The P4 control block `Ctrl` in the example below instantiates an action selector `as` that can contain at most 128 member entries. The action selector uses a `crc16` algorithm with output width of 10 bits to select a member entry within a group.

Table `indirect_with_selection` uses this instance by specifying the `psa_implementation` table property as shown. The control plane can add member and group entries to `as`. Each member can specify either a `foo` or `NoAction` action. When programming the table entries, the control plane **does not** include the fields of match type `selector` in the match key. The selector match fields are instead used to compose a list that is passed to the action selector instance. In the example below, the list `{hdr.ipv4.src_address, hdr.ipv4.protocol}` is passed as input to the `crc16` hash algorithm used for dynamic member selection by action selector `as`.

```
control Ctrl(inout H hdr, inout M meta) {  
  
    action foo() { meta.foo = 1; }  
  
    ActionSelector(PSA_HashAlgorithm_t.CRC16, 128, 10) as;  
  
    table indirect_with_selection {  
        key = {  
            hdr.ipv4.dst_address: exact;  
            hdr.ipv4.src_address: selector;  
            hdr.ipv4.protocol: selector;  
        }  
        actions = { foo; NoAction; }  
        psa_implementation = as;  
    }  
  
    apply {  
        indirect_with_selection.apply();  
    }  
}
```

Note that the management of action selector entries in the presence of link failures is outside the scope of the PSA. Fast-failover requires information from the control plane and will be addressed as part of the P4Runtime API^[2 - Chapter 4] working group.

8.13. Timestamps

A PSA implementation provides an `ingress_timestamp` value for every packet in the `Ingress` control block, as a field in the struct with type `psa_ingress_input_metadata_t`. This timestamp should be close to the time that the first bit of the packet arrived to the device, or alternately, to the time that the device

began parsing the packet. This timestamp is *not* automatically included with the packet in the **Egress** control block. A P4 program wishing to use the value of **ingress_timestamp** in egress code must copy it to a user-defined metadata field that reaches egress.

A PSA implementation also provides an **egress_timestamp** value for every packet in the **Egress** control block, as a field of the struct with type **psa_egress_input_metadata_t**.

One expected use case for timestamps is to store them in tables or **Register** instances to implement checking for timeout events for protocols, where precision on the order of milliseconds is sufficient for most protocols.

Another expected use case is INT (In-band Network Telemetry^[1]), where precision on the order of microseconds or smaller is necessary to measure queueing latencies that differ by those amounts. It takes only 0.74 microseconds to transmit a 9 Kbyte Ethernet jumbo frame on a 100 gigabit per second link.

For these applications, it is recommended that an implementation's timestamp increments at least once every microsecond. Incrementing once per clock cycle in an ASIC or FPGA implementation would be a reasonable choice. The timestamp should increment at a constant rate over time. For example, it should not be a simple count of clock cycles in a device that implements dynamic frequency scaling^[2].

Timestamps are of type **Timestamp_t**, which is type **bit<W>** for a value of **W** defined by the implementation. Timestamps are expected to wrap around during the normal passage of time. It is recommended that an implementation pick a rate of advance and a bit width such that wrapping around occurs at most once every hour. Making the wrap time this long (or longer) makes timestamps more useful for several use cases.

- Checking for timeouts of protocol hello / keep-alive traffic that is on the order of seconds or minutes.
- If timestamps are placed into packets without converting them to other formats, then external data analysis systems using those timestamps will in many cases need to do so, e.g. to compare timestamps stored in packets by different PSA devices. These systems will need different formulas and/or parameters to perform this conversion for each wrap period, or to add extra external time references to the recorded data. The extra data required for accurate conversion is lower, and the likelihood of conversion mistakes is lower, if the timestamp values wrap less often.
- If timestamps are converted to other formats within a P4 program, it will need access to parameters that are likely to change every wrap time, e.g. at least a "base value" to add some calculated value to. A straightforward way to do this requires the control plane to update these values at least once or twice per timestamp wrap time.
- Programs that wish to use (**egress_timestamp** - **ingress_timestamp**) to calculate the queueing latency experienced by a packet need the wrap time to exceed the maximum queueing latency.

Examples of the number of bits required for wrap times of at least one hour:

- A 32-bit timestamp advancing by 1 per microsecond takes 1.19 hours to wrap.
- A 42-bit timestamp advancing by 1 per nanosecond takes 1.22 hours to wrap.

A PSA implementation is not required to implement time synchronization, e.g. via PTP^[3] or NTP^[4].

The control plane API excerpt below is intended to be added as part of the P4Runtime API.

```
// The TimestampInfo and Timestamp messages should be added to the
// "oneof" inside of message "Entity".

// TimestampInfo is only intended to be read. Attempts to update this
// entity have no effect, and should return an error status that the
// entity is read only.
```

```

message TimestampInfo {
    // The number of bits in the device's `Timestamp_t` type.
    uint32 size_in_bits = 1;
    // The timestamp value of this device increments
    // `increments_per_period` times every `period_in_seconds` seconds.
    uint64 increments_per_period = 2;
    uint64 period_in_seconds = 3;
}

// The timestamp value can be read or written. Note that if there are
// already timestamp values stored in tables or `Register` instances,
// they will not be updated as a result of writing this timestamp
// value. Writing the device timestamp is intended only for
// initialization and testing.

message Timestamp {
    bytes value = 1;
}

```

For every packet **P** that is processed by ingress and then egress, with the minimum possible latency in the packet buffer, it is guaranteed that the **egress_timestamp** value for that packet will be the same as, or slightly larger than, the **ingress_timestamp** value that the packet was assigned on ingress. By "slightly larger than", we mean that the difference (**egress_timestamp** - **ingress_timestamp**) should be a reasonably accurate estimate of this minimum possible latency through the packet buffer, perhaps truncated down to 0 if timestamps advance more slowly than this minimum latency.

Consider two packets such that at the same time (e.g. the same clock cycle), one is assigned its value of **ingress_timestamp** near the time it begins parsing, and the other is assigned its value of **egress_timestamp** near the time that it begins its egress processing. It is allowed that these timestamps differ by a few tens of nanoseconds (or by one "tick" of the timestamp, if one tick is larger than that time), due to practical difficulties in making them always equal.

Recall that the binary operators **+** and **-** on the **bit<W>** type in P4 are defined to perform wrap-around unsigned arithmetic. Thus even if a timestamp value wraps around from its maximum value back to 0, you can always calculate the number of ticks that have elapsed from timestamp t_1 until timestamp t_2 using the expression $(t_2 - t_1)$ (if more than 2^W ticks have elapsed, there will be aliasing of the result). For example, if timestamps were $W \geq 4$ bits in size, $t_1 = 2^W - 5$, and $t_2 = 3$, then $(t_2 - t_1) = 8$. There is thus no need for conditional execution to calculate such elapsed times.

It is sometimes useful to minimize storage costs by discarding some bits of a timestamp value in a P4 program for use cases that do not need the full wrap time or precision. For example, an application that only needs to detect protocol timeouts with an accuracy of 1 second can discard the least significant bits of a timestamp that change more often than every 1 second.

Another example is an application that needed full precision of the least significant bits of a timestamp, but the combination of the control plane and P4 program are designed to examine all entries of a **Register** array where these partial timestamps are stored more often than once every 5 seconds, to prevent wrapping. In that case, the P4 program could discard the most significant bits of the timestamp so that the remaining bits wrap every 8 seconds, and store those partial timestamps in the **Register** instance.

8.14. Packet Digest

A digest is one mechanism to send a message from the data plane to the control plane. Another is to send a packet to the control plane via the port numbered **PSA_PORT_CPU**. Sending a packet to port

`PSA_PORT_CPU` typically sends most or all of the original packet headers, and perhaps also the payload, each as a separate message to be received and processed by the control plane. The contents of a digest for one packet are typically much smaller than the packet. A PSA implementation can take advantage of this, e.g. it might combine digests for multiple packets into larger messages, to reduce the rate of messages sent to the control plane.

A digest message may contain any values from the data plane. Because a P4 program may have multiple Digest instances, each with different message contents, the PSA implementation as a whole must provide the ability to distinguish the messages created by different Digest instances from each other.

In PSA, a digest is created by calling the `pack` method on the digest instance. The argument is the value to be included in the digest, often a collection of values in a P4 `struct` type. The compiler decides the best serialization format to send the digest contents to a local software agent, which is responsible for sending the digest data in a form defined by the P4Runtime API specification.

A PSA program can instantiate multiple Digest instances in the same IngressDeparser control block, and make at most one `pack` call on each instance during a single execution of this control block. A PSA implementation need not support the use of the Digest extern in the EgressDeparser control block.

There is no requirement that if multiple Digest messages are created while processing the same packet, that these messages must be "bundled together" in any way. An implementation is free to put them in separate queues per Digest instance, for example, and they may arrive to the controller completely separate from each other, and in a different order than they were generated. It is recommended that a PSA implementation send Digest messages *from a single Digest instance* to the control plane in the order they were generated.

If you wish to associate multiple Digest messages from different instances with each other in control plane software, it may suit your purposes to include a common sequence number or timestamp in all Digest messages generated by the same packet. Then use those in the control plane for correlation of different messages.

Since high speed PSA implementations are expected to be able to generate digests much faster than control software can consume them, it is expected that loss of such digest messages will occur if the data plane generates them too quickly. It is recommended that PSA implementations maintain a count of digest messages that the data plane creates, but do not reach the control plane, independently for each digest instance.

```
extern Digest<T> {
    Digest();                               /// define a digest stream to the control plane
    void pack(in T data);                   /// emit data into the stream
}
```

Below is a part of an example program that demonstrates using a digest to notify the control plane about source Ethernet MAC addresses and ingress ports of packets that have not been seen before.

```
struct mac_learn_digest_t {
    EthernetAddress srcAddr;
    PortId_t        ingress_port;
}

struct metadata {
    bool            send_mac_learn_msg;
    mac_learn_digest_t mac_learn_msg;
}

// This is part of the functionality of a typical Ethernet learning bridge.
```

```

// The control plane will typically enter the _same_ keys into the
// learned_sources and l2_tbl tables. The entries in l2_tbl are searched for
// the packet's dest MAC address, and on a hit the resulting action tells
// where to send the packet.

// The entries in learned_sources are the same, and the action of every table
// entry added is NoAction. If there is a _miss_ in learned_sources, we want
// to send a message to the control plane software containing the packet's
// source MAC address, and the port it arrived on. The control plane will
// make a decision about creating an entry with that packet's source MAC
// address into both tables, with the l2_tbl sending future packets out this
// packet's ingress_port.

// This is only a simple example, e.g. there is no implementation of
// "flooding" shown here, typical when a learning bridge gets a miss when
// looking up the dest MAC address of a packet.

control ingress(inout headers hdr,
                inout metadata meta,
                in  psa_ingress_input_metadata_t istd,
                inout psa_ingress_output_metadata_t ostd)
{
    action unknown_source () {
        meta.send_mac_learn_msg = true;
        meta.mac_learn_msg.srcAddr = hdr.ethernet.srcAddr;
        meta.mac_learn_msg.ingress_port = istd.ingress_port;
        // meta.mac_learn_msg will be sent to control plane in
        // IngressDeparser control block
    }

    table learned_sources {
        key = { hdr.ethernet.srcAddr : exact; }
        actions = { NoAction; unknown_source; }
        default_action = unknown_source();
    }

    action do_L2_forward (PortId_t egress_port) {
        send_to_port(ostd, egress_port);
    }

    table l2_tbl {
        key = { hdr.ethernet.dstAddr : exact; }
        actions = { do_L2_forward; NoAction; }
        default_action = NoAction();
    }

    apply {
        meta.send_mac_learn_msg = false;
        learned_sources.apply();
        l2_tbl.apply();
    }
}

control IngressDeparserImpl(packet_out packet,
                            out empty_metadata_t clone_i2e_meta,
                            out empty_metadata_t resubmit_meta,
                            out empty_metadata_t normal_meta,
                            inout headers hdr,
                            in metadata meta,
                            in psa_ingress_output_metadata_t istd)
{
    CommonDeparserImpl() common_deparser;
    Digest<mac_learn_digest_t>() mac_learn_digest;
    apply {
        if (meta.send_mac_learn_msg) {
            mac_learn_digest.pack(meta.mac_learn_msg);

```

```
    }  
    common_deparser.apply(packet, hdr);  
}  
}
```

- [1] <http://p4.org/p4/inband-network-telemetry>
- [2] https://en.wikipedia.org/wiki/Dynamic_frequency_scaling
- [3] https://en.wikipedia.org/wiki/Precision_Time_Protocol
- [4] https://en.wikipedia.org/wiki/Network_Time_Protocol

9. Atomicity of control plane API operations

All table add, delete, and modify operations must be atomic relative to packet forwarding. That is, for every table **apply** operation, and every control plane operation on a table that adds, deletes, or modifies one table entry, the **apply** operation should behave as if that control plane operation has not yet occurred, or as if the control plane operation is complete. The P4 program should never behave as if the control plane operation is partially complete.

Note that this requirement is for every table **apply** operation individually. A PSA implementation is not required to support performing multiply **apply** operations on the same table in the same invocation of a control block. If it does support that, it is allowed that a control plane update *may* occur after one **apply** call by a packet to a table, but before the next **apply** call by the same packet.

A PSA implementation should give an error and fail to compile P4 programs for which it cannot meet this atomicity requirement. For example, perhaps the implementation can only satisfy this requirement for tables with actions having at most 128 bits of action parameters, and thus gives an error if you attempt to compile a P4 program that contains an action with more bits of parameters.

For example, suppose a table **T** has an action **A** with 100 total bits of action parameters, and the control plane has added a table entry with a search key **K** and action **A**. Later the control plane performs an update operation on the entry with key **K** that leaves the key **K** the same, but changes the 100 bits of action parameters. Every packet doing an **apply** on table **T** and matching the entry with key **K** should execute action **A** with either the old 100 bits of action parameters, or the new 100 bits of action parameters.

The P4Runtime API enables controllers to create "batch" messages that perform more than one single operation, as defined here. If so, a PSA implementation need only ensure that each single operation is atomic. There is no requirement that a sequence of *multiple* table entry add, delete, or update operations should be atomic.

The same applies for all control plane API operations on externs, unless the control plane operation explicitly documents otherwise.

In particular, ActionProfile and ActionSelector single operations, such as adding a member to a group, removing a member from a group, adding an empty group, deleting an empty group, or modifying the action parameters of an action added earlier to a group, should all be atomic.

Also, a control plane read, or write, of a single element of a Register array should be atomic, and behave as if it occurred before or after (but not during) any P4 program's section of code labeled with the **@atomic** annotation. There is *no* control plane operation on a Register that can atomically read an element, then write back a modified value.

Advice for P4 developers: If you desire a capability for the control plane to atomically read, modify, then write back a Register array element, you should write your P4 program such that the desired read, modify, and write operation can be done by a packet that your control plane can inject into the data plane, e.g. via packet in / packet out P4Runtime API operations.

A high speed PSA implementation might process hundreds or thousands of packets between each single control plane operation. There are common "write tables from later to earlier in the data flow", sometimes also called "back to front" or "pointer flipping", techniques used by existing control planes to achieve an effect that is similar to making a sequence of many table entry operations atomic relative to packet forwarding. Recent research analyzes these techniques in a more general setting^[1].

[1] Pavol Cerny, Nate Foster, Niles Jagnik, and Jedidiah McClurg, "Consistent Network Updates in Polynomial Time". International Symposium on Distributed Computing (DISC), Paris, France, September 2016.

Appendix A: Open Issues

As with any work in progress, we have a number of open issues that are under discussion in the working group. In addition to the TBDs in the document, there are a number of larger issues that are summarized here:

A.1. Action Selectors

The size parameter in the `action_selector` instance that defines the maximum number of members in a selector. In some cases it might be useful to allow the controller to dynamically provision resources on the selector or to utilize different selector sizes on different targets, while using a common P4 program.

We also need to formalize the interaction of action profiles and action selectors with counters and meters.

A.2. Observation and control of congestion

The current PSA does not provide any mechanisms to observe if particular output ports or queues are leading to congestion in the packet buffer. Thus it is not possible without using mechanisms defined outside of PSA to implement a feature like Explicit Congestion Notification (ECN)^[1]. One possibility here is to define a small field, perhaps only 1 bit, that is part of the metadata associated with each packet as it begins egress processing. This field would indicate "how much congestion" the packet experienced in the packet buffer.

There is also currently no way defined in PSA for ingress P4 code to send information about a packet to the packet buffer that might influence the behavior of a congestion control algorithm, such as Approximate Fair Drop (AFD). This is partly because of the variety of congestion control mechanisms in use by switches today.

It would be desirable to define in PSA a small set of fields about a packet that would be useful inputs to multiple congestion control algorithms. One possibility is a hash of the packet's "flow id", often implemented as a hash of packet header fields like IP source and destination address, IP protocol, and optionally TCP/UDP source and destination ports. Given that P4 programmable devices can implement network protocols other than IP, including custom ones, a more general mechanism is desirable in PSA devices.

A.3. Enabling full implementation of In-band Network Telemetry

One promising use case for P4 programmable network devices is to implement In-band Network Telemetry^[1 - Chapter 8]. While PSA mechanisms such as timestamps enable a significant portion of INT features to be implemented, they do not yet define any mechanisms to access information such as egress port link utilization or queue occupancy^{footnote:psaint}[<https://github.com/p4lang/p4-spec/issues/510>].

A.4. PSA profiles

We are considering whether to specify different limits that a certain PSA implementation has to have in order for the implementation to be considered compliant. The main point of PSA is to enable a variety of devices, and thus limits may be artificial. On the other hand, for most interesting applications, it is necessary to support a minimum of functionality.

Appendix B: Implementation of the InternetChecksum extern

Besides RFC 1461, RFC 1071 and RFC 1141 also contain useful tips on efficiently computing the Internet checksum, especially in software implementations.

Here we give reference implementations for the methods of the `InternetChecksum` extern, specified with the syntax and semantics of $P4_{16}$, with extensions of a `for` loop and a `return` statement for returning a value from a function.

The minimum internal state necessary for one instance of an `InternetChecksum` object is a 16-bit bit vector, here called `sum`.

```
// This is one way to perform a normal one's complement sum of two
// 16-bit values.
bit<16> ones_complement_sum(in bit<16> x, in bit<16> y) {
    bit<17> ret = (bit<17>) x + (bit<17>) y;
    if (ret[16:16] == 1) {
        ret = ret + 1;
    }
    return ret[15:0];
}

bit<16> sum;

void clear() {
    sum = 0;
}

// Restriction: data is a multiple of 16 bits long
void add<T>(in T data) {
    bit<16> d;
    for (each 16-bit aligned piece d of data) {
        sum = ones_complement_sum(sum, d);
    }
}

// Restriction: data is a multiple of 16 bits long
void subtract<T>(in T data) {
    bit<16> d;
    for (each 16-bit aligned piece d of data) {
        // ~d is the negative of d in one's complement arithmetic.
        sum = ones_complement_sum(sum, ~d);
    }
}

// The Internet checksum is the one's complement _of_ the one's
// complement sum of the relevant parts of the packet. The methods
// above calculate the one's complement sum of the parts in the
// variable 'sum'. get() returns the bitwise negation of 'sum', which
// is the one's complement of 'sum'.

bit<16> get() {
    return ~sum;
}

bit<16> get_state() {
    return sum;
}
```

```

}

void set_state(bit<16> checksum_state) {
    sum = checksum_state;
}

```

Appendix C: Example implementation of Counter extern

The example implementation below, in particular the function `next_counter_value`, is not intended to restrict PSA implementations. The storage format for `PACKETS_AND_BYTES` type counters demonstrated there is one example of how it could be done. Implementations are free to store state in other ways, as long as the control plane API returns the correct packet and byte count values.

Two common techniques for counter implementations in the data plane are:

- wrap around counters
- saturating counters, that 'stick' at their maximum possible value, without wrapping around.

This specification does not mandate any particular approach in the data plane. Implementations should strive to avoid losing information in counters. One common implementation technique is to implement an atomic "read and clear" operation in the data plane that can be invoked by the control plane software. The control plane software invokes this operation frequently enough to prevent counters from ever wrapping or saturating, and adds the values read to larger counters in driver memory.

```

Counter(bit<32> n_counters, PSA_CounterType_t type) {
    this.num_counters = n_counters;
    this.counter_vals = new array of size n_counters, each element with type W;
    this.type = type;
    if (this.type == PSA_CounterType_t.PACKETS_AND_BYTES) {
        // Packet and byte counts share storage in the same counter
        // state. Should we have a separate constructor with an
        // additional argument indicating how many of the bits to use
        // for the byte counter?
        W shift_amount = TBD;
        this.shifted_packet_count = ((W) 1) << shift_amount;
        this.packet_count_mask = (~(W) 0) << shift_amount;
        this.byte_count_mask = ~this.packet_count_mask;
    }
}

W next_counter_value(W cur_value, PSA_CounterType_t type) {
    if (type == PSA_CounterType_t.PACKETS) {
        return (cur_value + 1);
    }
    // Exactly which packet bytes are included in packet_len is
    // implementation-specific.
    PacketLength_t packet_len = <packet length in bytes>;
    if (type == PSA_CounterType_t.BYTES) {
        return (cur_value + packet_len);
    }
    // type must be PSA_CounterType_t.PACKETS_AND_BYTES
    // In type W, the least significant bits contain the byte
    // count, and most significant bits contain the packet count.
    // This is merely one example storage format. Implementations
    // are free to store packets_and_byte state in other ways, as

```

```

// long as the control plane API returns the correct separate
// packet and byte count values.
W next_packet_count = ((cur_value + this.shifted_packet_count) &
                      this.packet_count_mask);
W next_byte_count = (cur_value + packet_len) & this.byte_count_mask;
return (next_packet_count | next_byte_count);
}

void count(in S index) {
    if (index < this.num_counters) {
        this.counter_vals[index] = next_counter_value(this.counter_vals[index],
                                                       this.type);
    } else {
        // No counter_vals updated if index is out of range.
        // See below for optional debug information to record.
    }
}

```

Optional debugging information that may be kept if an `index` value is out of range includes:

- Number of times this occurs.
- A FIFO of the first N out-of-range index values that occur, where N is implementation-defined (e.g. it might only be 1). Extra information to identify which `count()` method call in the P4 program had the out-of-range `index` value is also recommended.

Appendix D: Rationale for design

D.1. Why egress processing?

Question: Why is it useful to have separate ingress vs. egress processing in a switch device?

There have been packet processing ASICs built that effectively only do 'ingress' processing, then go to a packet buffer with one or more queues, and then go out of the device, effectively being restricted to no or "empty" egress processing.

There are a few things that are trickier to do in such a device.

1. Last-nanosecond changes to the packet

If you want to measure the queuing latency through the device, and put a measurement of this quantity inside the packet somewhere, it is in general not possible to know the queueing latency before the packet is sent to the packet buffer. There are *special cases* where you can predict it, e.g. when there is a single FIFO queue feeding a constant bit rate output port, with nothing like Ethernet pause flow control.

But if you have variable bit rate links, e.g. because of things like Ethernet pause flow control, or Wi-Fi signal quality changes, or if you have multiple class-of-service queues with a scheduling policy between them like weighted fair queueing, then it is not possible to predict at the time the packet is enqueued, when it will be dequeued. The queueing latency depends upon unknown future events, such as whether Ethernet pause frames will arrive, or how many and what size of packets arriving in the near future will be put into which class of service queues for the same output port.

In such cases, having egress processing for taking the measurement, after it is known and easy to calculate as "dequeue time - enqueue time", allows the egress processing to modify the packet further.

2. Multicast efficiency and flexibility

It is possible in a PSA device to handle multicast by doing a recirculate plus clone operation for each of N copies to be made, but this reduces the processing capacity of ingress that is available to newly arriving packets, in particular newly arriving packets that you might consider more important to keep than the multicast packets.

By designing a packet buffer that can take a packet with a 'multicast group id', which the control plane configures to make copies to a selected set of output ports, it frees up the part of the system that performs ingress processing to accept new packets more quickly, and at a more predictable rate.

There could still be a challenge in designing the packet replication portion of the system not to fall behind when many multicast packets to be replicated to many output ports arrive close together in time, but it is fairly easy to separate the concerns of multicast from unicast packets. For example, a device implementer could prioritize unicast packets so that they are not slowed down if multicast replication is falling behind.

Once you have multicast designed in this way, there are still multicast use cases where one needs to process different copies of the packet differently. For example, the copy going out port 5 might need a VLAN tag of 7 placed in its header, whereas the copy going out port 2 might need a VLAN tag of 18 placed in its header. Similarly for multicast packets entering one of many flavors of tunnels, e.g. VXLAN, GRE, etc. By doing this per-copy modifications in egress processing, the packet replication logic can be kept very simple—just make identical copies of the packet as ingress finished with it, except for some kind of unique 'id' on each copy that egress processing can use to distinguish them.

D.2. No output port change during egress

Question: Why can't my P4 program change the output port during egress processing?

In a network device that has many input and output ports, packets can arrive at or near the same time on multiple input ports, all destined for the same output port.

Packet buffers are typically designed into such network devices, to store the packets that cannot be sent out immediately, absorbing this short term congestion.

For a given output port P, we now wish to retrieve packets from the packet buffer at a rate that is equal to the rate we will send them to port P, typically equal to the maximum bit rate that it is possible to send data out of port P.

Packet scheduling algorithms such as weighted fair queueing, and many others, have been developed that can determine which among a set of potentially many FIFO queues that a packet should be read from next, and sent out on the port.

These link scheduling algorithms are real time algorithms with very tight timing constraints. If they go too slow, the output port goes idle and its capacity is wasted. If they go too fast, we read packets from the packet buffer faster than they can be transmitted on the port, and we are back at the same problem we had originally—either drop some of the packets, or store them somewhere again until the port is ready to transmit them.

Such a scheduling algorithm that handles multiple output ports must know which output port all packets are destined to, before they are put into the packet buffer. If that target output port can be changed after the packet is read out, then we can simultaneously overload one output port while starving another.

That is why the `egress_port` of a packet must be selected during ingress processing, and egress processing is not allowed to change it.

These scheduling algorithms also need to know the size of each packet, i.e. the size as it will be when transmitted on the port.

It is possible in egress P4 code to drop a packet, or to change the size of the packet by adding or removing headers. Very likely P4-programmable network devices will have their scheduling algorithms run just slightly faster than the port to handle cases where many packets in a row are decreased in size during egress processing, and have tight control loops monitoring the size of packets leaving egress processing to make small adjustments in the rate that the scheduling algorithm operates for each port. Either that, or they will just leave some fraction of the output port's capacity unused during times when all packet sizes are being decreased.

Certainly if there are long durations of time when egress decides to drop all packets to an output port, that port will go idle. The scheduling algorithm implementations are all built with a finite maximum packet scheduling rate.

D.3. Ingress deparser and egress parser

Question: P4₁₄ did not have an ingress deparser, or an egress parser. Why does PSA have these things?

P4₁₄ did not have these things explicitly, but there was also not much *explicitly* stated in the P4₁₄ specification about what data about each packet was carried from ingress to egress. Often such things were left implicit. Some implementations have an ingress deparser whose order of emitting headers is auto-generated from the P4₁₄ program's parser code. This leads to restrictions on your P4₁₄ program, not stated in the P4₁₄ specification, that the contents of your headers and metadata must be in a state where if you deparse at that point, then your P4₁₄ parser code must be able to parse that packet, or else the device will fail to parse it in the (implicit) egress parser.

By making the ingress deparser and egress parser explicit, we hope to make this behavior more defined, and more portable across different PSA implementations. We expect that the common case will be that the ingress and egress parsers will have much (or all) code in common with each other, and this is easy to do using P4₁₆'s capability for one parser to call another, i.e. you can write a common parser, then call it from your ingress and egress parsers.

You can also choose to make the two parsers different, and have full control over the differences between them. For example, you might wish your egress parser to handle extra headers that you only put onto cloned packets, that should never appear on packets from input ports.

Similarly for the ingress deparser. By making this an explicit and separate control block, you now have full control over exactly what data about a packet is included when it is sent to the packet buffer, and what is not. In P4₁₄, it was implicit that "all packet metadata used somewhere in egress code" was carried along with each packet. This can still be done in P4₁₆ programs for the PSA, but now you must be explicit in doing so. With PSA, you now have a way to restrict how much data is carried with each packet, which can be important if the I/O bandwidth of the packet buffer becomes a bottleneck.

Appendix E: Multi-pipeline PSA devices

The highest packet rate network devices today are ASICs running at a clock rate on the order of 1 to 2 GHz. This discussion will assume 1 GHz for the sake of a concrete numerical example, but everything discussed here scales linearly with the clock rate.

It is common to design a portion of a network ASIC such that it can start processing a new packet once every clock cycle, and finish a packet every clock cycle. The latency might be hundreds of clock cycles from starting a packet until it is complete. P4 tables in such ASICs are typically implemented using logic such as TCAMs and SRAMs. TCAM designs can do 1 search per clock cycle. The lowest area and power SRAMs can do 1 read or 1 write per clock cycle.

While there are "multi ported" SRAM designs that can be read and/or written multiple times per clock cycle, these have a noticeable increase in area and power over the "single ported" designs that are limited to 1 access per cycle. If multi ported TCAM designs even exist, the cost premium for multi-ported TCAMs

is likely to be even higher than the cost premium for multi-ported SRAMs. The typical way to achieve higher TCAM search rates is via parallelism, by creating multiple copies of the desired TCAM, which is a linear increase in area and power (at least).

Due to these issues, if one wishes to create a switch ASIC that achieves a packet processing rate of N times the clock rate, e.g. 2 billion packets per second in a 1 GHz ASIC, the most straightforward way to do this is to take advantage of the fact that packet processing is (mostly) an embarrassingly parallel problem^[1], and create a device with N pipelines^[2], each pipeline processing packets at 1 billion packets per second.

A PSA device designed in this way would typically have N ingress pipelines, plus N egress pipelines. It is common to assign multiple physical ports of the switch ASIC to each pipeline in a hard-wired fashion, e.g. a device with 32 100 Gigabit Ethernet ports might physically hard-wire ports 0 through 15 to pipeline 0, and ports 16 through 31 to pipeline 1. All packets received on ports 0 through 15 will be processed by ingress pipeline 0, then go to the packet buffer (if they were not dropped in ingress), then go to egress pipeline 0 if ingress chose output port 0 through 15, or go to egress pipeline 1 if ingress chose output port 16 through 31.

In such a device, typically you will want the same P4 program to be run on every one of these N ingress pipelines, and every one of the N egress pipelines.

It is physically possible in such a device for the control plane to install different table entries in the different pipelines, and there are use cases where this can help achieve higher scale in number of usable table entries. For example, perhaps you have an ingress table with the ingress port as one of the fields in its search key. If this is the case, the packet processing behavior is the same even if table entries for port X are installed only in the ingress pipeline that processes packets from port X . Installing that table entry in the other pipelines would be an unnecessary use of table space, since it could never be matched. Whether the device-dependent control software of such a PSA device enables taking advantage of such space savings is implementation dependent.

Regardless of this issue, applying tables in P4 is an embarrassingly parallel activity, because as long as table entries are installed where they might be matched, pipelines can operate completely independently of each other with no communication between them (one small exception is described below). The same is true for most PSA externs, e.g. `ActionProfile`, `ActionSelector`, `Checksum`, `Digest`, `Hash`, and `Random`. What P4 tables and these externs have in common is: either the P4 program behavior cannot modify their state at all (e.g. tables, `ActionProfile`, `ActionSelector`), or can only modify it in a way that does not affect the processing of other packets (e.g. `Checksum`, `Digest`, `Hash`). `Random` is a special case here: updating the state of a pseudo-random number generator can affect the processing of other packets, but this is typically not a concern for the way that pseudo-random numbers are used (e.g. randomly choosing packets to drop or mark in Random Early Detection).

For counters, there is counter state maintained independently in each pipeline, but if corresponding counter entries in each pipeline count "the same thing" (e.g. packets matching a table entry with key X installed in all pipelines), then it is straightforward to add up the corresponding counter values from all pipelines.

Consider a device where meter state is maintained independently in each pipeline. If you have a multi-pipeline PSA device, and wish to achieve the effect: "meter all packets matching table entry X to at most Y bytes per second", this is *not* an embarrassingly parallel problem. It could be done by coordinating state between the pipelines, e.g. using something like cache coherency protocols commonly implemented within multi-core CPUs, or by "moving the packets to where the shared state is", e.g. recirculating packets to a common pipeline where the meter state is kept. Both of these techniques lead to lower packet processing performance, at least in some cases, and both add complexity to the system. It is typical in network switches to simply maintain the meter state independently in each pipeline and not coordinate it, and accept the resulting behavior.

This issue is not specific to packet switches. It falls under the category of accessing mutable state in a distributed system, with not only correctness concerns, but very strong performance concerns.

The **Register** extern is more general in its capability than meters, and the same potential issue of state being split across multiple pipelines exists. It is recommended that you talk to your PSA device vendor about this issue if it could affect features you wish to write in your P4 program. If a PSA device does not implement automatic coherency for such state, common strategies are the same as mentioned above for meters: accept the resulting behavior of the independently maintained state in each pipeline, or move the relevant packets to one place where the state is maintained.

Note that the proposed **psa_idle_timeout** table property introduces a way by which doing table **apply** operations *does* update state within a P4 table. Each table entry requires at least one, and more likely several, bits of state to represent a "last matched time" value, and this value is updated with every **apply** operation. If this state in tables with this option is not automatically coordinated between pipelines, then it can differ for corresponding table entries in different pipelines. An entry with key X in one pipeline could remain unmatched for longer than the desired timeout, at the same time that the corresponding entries with key X are recently matched in other pipelines. One possible approach to handle this is for the PSA device and its implementation-specific control software to make the existence of multiple pipelines explicit to the control plane software in some way, e.g. assign each pipeline, and the tables and externs it contains, a distinct name.

For programming multiple pipelines, it is the responsibility of the vendor and of target dependent tools to specify how PSA programs are mapped to multiple pipelines. An implementation may use a copy of the PSA program on each pipeline, thus keeping pipelines fully isolated.

[1] https://en.wikipedia.org/wiki/Embarrassingly_parallel

[2] Here and elsewhere in P4 specification documents, the term "pipeline" refers to a portion of a P4 implementation that implements, for example, the behavior of IngressParser, Ingress, then IngressDeparser in PSA. This is by now traditional in P4 specifications, and although we will not try to change that here, note that there are other reasonable hardware designs that can accomplish this goal that few would call a pipeline, e.g. a collection of CPU cores running in parallel, each processing different packets.

Appendix F: Packet ordering

This section describes *recommendations* for PSA implementations on the order that packets are processed. These are not requirements, since there are known implementation techniques, especially parallelism that can be taken advantage of in a variety of ways, that can lead to cheaper implementations if these recommendations are not followed. We recommend that developers selecting P4 devices ask their designers about these issues, if those developers consider the issues important for their purposes.

Recommendation 1: Packets that arrive on the same input port should begin ingress processing in the same relative order as they arrived.

Recommendation 2: Packets that go out on the same output port should be transmitted on the port in the same relative order that they began egress processing.

Recommendation 3: PRE unicast packets (i.e. those that follow the "enqueue one packet" path in the pseudocode of [Section 7.3](#)) that arrived from the same ingress port, did ingress processing once (i.e. were not resubmitted or recirculated), were sent to the PRE with the same `class_of_service` value, and destined for the same egress port, should begin egress processing in the same relative order as they began ingress processing.

It is expected that some PSA implementations will implement the class of service mechanism by having a separate FIFO queue per class of service, and thus while unicast packets with the same ingress port, egress port, and class of service will pass through the system in FIFO order if they follow all recommendations above, unicast packets with the same ingress and egress port, but different classes of service, may be processed by the egress control block in a different order than they were processed by the ingress control block.

If an implementation satisfies recommendations 1 through 3, then unicast traffic assigned to the same class of service will maintain its relative order through the device.

Recommendation 4: Consider PRE multicast packets (i.e. those that follow the "Make 0 or more copies" path in the pseudocode of [Section 7.3](#)) that arrived on the same ingress port, did ingress processing once, and were sent to the PRE with the same pair of values (`class_of_service`, `multicast_group`). For copies of those original packets that are destined to the same egress port, and with the same pair of values for (`egress_port`, `instance`), those copies should begin egress processing in the same relative order as the original packets began ingress processing.

There is no such expectation for multicast packets with different `class_of_service` values, again because of separate queues in the PRE for different `class_of_service` values.

It is also understood that for a short period of time after the control plane modifies the set of copies to be made for a particular `multicast_group` value, that it may be especially difficult to satisfy Recommendation 4. That recommendation is intended to apply only when the set of copies to be made for the multicast group has remained unchanged for a period of time.

If an implementation satisfies recommendations 1, 2, and 4, then multicast traffic assigned to the same class of service will maintain its relative order through the device, when multicast group membership has been stable for long enough.

Note that there is no recommendation to enforce any relative ingress to egress processing order of unicast packets vs. multicast packets. Commonly used mechanisms for creating multicast copies in the PRE allow unicast packets to "go around" the packet replication logic, which is unnecessary for unicast packets, and thus change the relative order of such packets there. Also, it is common in packet buffers to use separate queues for multicast traffic versus unicast.

We give some motivations for these recommendations below.

1. Expectations of hosts

While the Internet Protocol does not have strong ordering requirements for sequences of packets sent by one host to another, there are still widely deployed implementations of TCP that lead to significantly degraded throughput when the network frequently delivers packets to the receiver in a different order than they were transmitted. While significant research and development has been done to mitigate this issue, e.g. later Linux TCP implementations (since 2011 when version 2.6.35 was released) are much more resilient to this problem than earlier versions, there are still many commonly deployed TCP implementations that suffer from this issue. See references within Kandula et al's work^[1] for some of the research done towards making TCP more robust in the face of network packet reordering.

Such TCP implementations interpret acknowledgements with repeated cumulative acknowledgement sequence numbers as a likely indication of packet loss in the network, and reduce their sending window in an effort to avoid network congestion.

While applications using UDP should also be aware of possible packet reordering in a network, some of them behave poorly if this reordering becomes common^[2].

These expectations of hosts are a significant reason why ECMP (Equal Cost Multi Path) path selection and LAG (Link Aggregation Group) link selection are so often done by using a hash of packet header fields such as IP source and destination address, and TCP or UDP source and destination port. Choosing among parallel paths in this way helps to preserve the order of packets in the same application flow, at the cost of not balancing the load as evenly as possible. If a network device's internal implementation reordered packets, it would be an independent source of network reordering.

2. Implementation of stateful protocols

This reason is much less significant than the previous one. We mention it here primarily so implementers of networking protocols are aware of the issue. This issue is only relevant for a relatively small fraction of network protocols.

Some networking protocols add sequence numbers to packets, and require either dropping packets that arrive out of sequence number order at a later network point (e.g. GRE with sequence numbers enabled), or with a looser check that allows some amount of network reordering to occur without dropping the packets (e.g. IPsec). When a device implementing these protocols does the sequence number insertion or checking in a different order than packets are sent or received on a physical port, that is effectively another kind of network reordering that can affect the performance of these protocols.

Similarly, there are some protocols like IP header compression, where multiple variants have been developed, some with more or less robustness in the face of network reordering.

[1] S. Kandula, D. Katabi, S. Sinha, and A. Berger, "Dynamic load balancing without packet reordering", ACM SIGCOMM Computer Communication Review, Vol. 37, No. 2, April 2007

[2] M. Laor and L. Gendel, "The effect of packet reordering in a backbone link on application throughput", IEEE Network, 2002.

Appendix G: Supporting empty action selector groups

As mentioned in [Section 8.12](#), a PSA data plane implementation need not implement any specific defined behavior if one attempts to add a table entry for a key that points at a group that is currently empty.

Some P4 users have expressed an interest in enabling a P4Runtime client (hereafter called the controller) to remove the last member of an action selector group, and have the data plane behave in a predictable way.

For example, if one has a table that maps logical interface id numbers to physical port numbers, using an action selector to implement LAG (Link Aggregation Group), what should the controller do when there was only one physical port in a LAG that was enabled, and that port goes down? A straightforward desired behavior from the controller's perspective is: issue the P4Runtime command to remove the last member from the group, and have an empty group action of "drop the packet" take effect for any packets applying the table and selecting the empty group.

Fully supporting empty action groups should meet all of these requirements:

- All P4Runtime API operations such as adding a member to a group (even when changing from empty to 1 member), removing a member from a group (even when changing from 1 member to empty), modifying the action associated with a member, etc. should be atomic relative to data packet processing. That is, every packet should be processed as if the table was in the old state, or the new state, with no undefined packet processing behavior.
- The empty group action that is executed when the matching table entry points at an empty group, may have an action name that is the same as, or different than, the action name used by the group when it was non-empty (if it was non-empty, and then became empty by removing its last member).

A high performance implementation will also be able to make changes to group membership using a number of data plane update operations that does not grow with the number of table entries that point at the group.

Achieving all of these requirements seems not to be possible with a PSA data plane implementation that meets only the minimum requirements for action selectors, i.e. one that restricts members of a group to all have the same action name, and that does not natively support predictable behavior if a group is empty in the data plane.

Below we describe one way that achieves the goals above, but only for a data plane implementation that supports multiple different action names in the same group at the same time. For a data plane that does not support this, the idea only *nearly* achieves the goal. It requires the empty group action to have the same action name as the non-empty groups of the action selector. This may be too onerous of a restriction on system developers to be worth implementing.

This behavior can be implemented via a small extra bit of logic in the P4Runtime server implementation (hereafter called the agent). The basic idea is that the agent obtains the empty group action, with action parameter values, e.g. from the compiled representation of the P4 program.

If no table entry currently "points at" an empty group G, then G's empty group action need not be installed anywhere in the data plane. Similarly if G is currently non-empty, and there are table entries currently pointing at G.

Suppose there is currently one member in G, and G is pointed at by at least one table entry. The controller issues a command to remove that only member from G.

The agent can implement this command by the following sequence of changes in the data plane:

1. Add to G a new member, which is the empty group action. Now in the data plane, for a short time, G contains two members.
2. Remove from G the member that the controller requested to be removed. Now the data plane version of G has only one member containing the empty group action, so all packets using G will execute that action.

When G is currently empty as far as the controller is concerned (but contains one member pointing at the empty group action in the data plane), and the controller adds one member to it, the agent can implement this via these steps:

1. Add to G the member requested by the controller. The data plane temporarily has two members for G, including the empty group action.
2. Remove from G the empty group action. Now G in the data plane is back to the one member that the controller wants.

A PSA implementation with an agent that implements empty action selector groups in this way must implement each of the two steps for such transitions in an atomic way, as described in [Chapter 9](#), but it is allowed for one or more packets to be processed in the intermediate state that exists between the two steps.

If a PSA implementation supports multiple different action names in the same group at the same time, then there is no need to read further. Below we only describe what might be done for a data plane that restricts each group to contain only actions with the same action name.

Because a PSA implementation need not support multiple different action types in the same action selector group at the same time (mentioned in [Section 8.12](#)), a developer wishing to take advantage of this in a portable way may need to modify one or more of the actions used in their tables that use action selectors.

For example, if in the LAG port selection example mentioned earlier, there was only one action for a table `lag` defined like this:

```

action set_output_port (PortId_t p) {
    user_meta.out_port = p;
}
ActionProfile(128) ap;
table lag {
    key = {
        // ... key fields go here ...
    }
    actions = { set_output_port; }
    psa_implementation = ap;
}

control cIngress (
    inout headers hdr,
    inout metadata user_meta,
    in    psa_ingress_input_metadata_t istd_meta,
    inout psa_ingress_output_metadata_t ostd_meta
) {
    apply {
        // ... earlier ingress code goes here ...
        lag.apply();
        send_to_port(ostd_meta, user_meta.out_port);
        // ... later ingress code goes here ...
    }
}

```

with a single action parameter equal to a physical port number of the device, one of the following approaches could be used, and of course there are likely to be other approaches not mentioned here.

- Approach 1: Use an invalid port number

Pick a value of type `PortId_t` that corresponds to no valid physical port of the device (TBD whether PSA should define such a value with a name—it currently doesn’t guarantee that such a value even exists for type `PortId_t`) and use that value for the empty group action of an empty group. In the code after `lag.apply`, add an `if` statement checking for that invalid value, like this:

```

apply {
    // ... earlier ingress code goes here ...
    lag.apply();
    if (user_meta.out_port == PORT_INVALID_VALUE) {
        ingress_drop(ostd_meta);
    } else {
        send_to_port(ostd_meta, user_meta.out_port);
    }
    // ... later ingress code goes here ...
}

```

- Approach 2: Add extra parameters to the action

In this case, add a 1-bit parameter indicating whether to drop the packet later. An `if` statement is still needed after applying the table.

```

action set_output_port (PortId_t p, bit<1> drop) {
    user_meta.out_port = p;
    user_meta.drop = drop;
}

// ...

apply {
    // ... earlier ingress code goes here ...
    lag.apply();
    if (user_meta.drop == 1) {
        ingress_drop(ostd_meta);
    } else {
        send_to_port(ostd_meta, user_meta.out_port);
    }
    // ... later ingress code goes here ...
}

```

In either case, an implementation might also support putting the `if` statement inside of the action `set_output_port`, but this is not required by PSA.

Appendix H: PSA Reserved Annotations

The following table shows PSA-specific reserved annotations. For P4 language reserved annotations such as `@tableonly` and `@defaultonly`, refer to the P4 language specification.

Annotation	Purpose	See Section
<code>@groupaction</code>	Action can only be used as an ActionSelector group action	Section 8.12

Appendix I: Revision History

I.1. Changes made in version 1.2

Version 1.2 was released December 22, 2022.

Summary of changes. For some of these, additional explanation is provided in sub-sections below.

- Add `@p4runtime_translation` annotations for all seven PSA numeric types in `psa.p4` (all changes in file `psa.p4`).
- Add clarifications on `@p4runtime_translation` annotation behavior ([Chapter 5](#)).
- Add documentation about the behavior of each `match_kind` ([Section 4.3](#)).
- Add explanation why there is no packet length field defined in PSA ([Section 7.5](#)).
- Add `match_kind optional` ([Section 4.3](#)).
- Changed `psa.p4` file explicitly so that it is *not* usable for compiling, by adding an `#error` preprocessor directive to it. Added a copy of `psa-for-bmv2.p4` from open source `p4c` compiler to `examples` directory for quick testing of correct syntax of example programs (all changes in file `psa.p4`).
- Remove control plane API function signatures from comments in file `psa.p4` (all changes in file `psa.p4`).
- Replace `@noWarnUnused` with standard `@noWarn("unused")` annotations in `psa.p4` (all changes in file `psa.p4`).
- Add `assert` and `assume` extern functions (all changes in file `psa.p4`).
- Clarifications of the behavior for different possible values of table property `psa_idle_timeout` ([Section 8.2.1](#)).

I.1.1. Add `match_kind optional`

This match kind was added to the vlmodel architecture in early 2020, and seems useful in that it (a) provides additional information about how the control plane plans to configure the matching rules of a table, and (b) the additional restrictions it has over match kind `ternary` provides opportunities for target devices to implement it more efficiently than `ternary`.

I.1.2. Remove control plane API function signatures

As a historical note, the control plane API function signatures that were formerly given in comments preceded by the string `@ControlPlaneAPI` were added very early in the process of writing version 1.0 of the PSA specification, and were never reviewed thoroughly. The P4Runtime API specification version 1.0 was being developed at the same time as version 1.0 of the PSA specification. They were removed because we believe that no one ever implemented precisely those APIs for any P4-programmable device (although similar APIs have been implemented), and their presence was confusing those new to the PSA specification, who believed that those function signatures were required to be implemented.

I.2. Changes made in version 1.1

Version 1.1 was released November 22, 2018.

Summary of changes. For some of these, additional explanation is provided in sub-sections below.

- Define numeric translation between P4Runtime API control plane and data plane values.
- Add the ability for packet clone sessions to create multiple copies.
- Add `psa_idle_timeout` table property.
- Add `psa_empty_group_action` table property.
- No longer require PSA implementation to support `Digest` extern instances in the egress pipeline.
- Several changes to the `psa.p4` include file.
- Several changes to example PSA programs in the `examples` directory.

I.2.1. Numeric translation between P4Runtime API values and data plane values

There was a series of meetings after PSA v1.0 was released to refine the details of the plan to do numeric translation of values with type `PortId_t` (and `ClassOfService_t`, and optionally other types in the future). PSA v1.1 reflects the latest design for how to accomplish this. Changes can be found here:

- Section 4.1 "PSA type definitions"
- Section 4.4 "Data plane vs. control plane data representations"

I.2.2. Add the ability for packet clone sessions to create multiple copies

In PSA v1.0, requesting to make a clone of a packet was restricted to creating a single clone, sent to a single output port. In PSA v1.1 you may now configure a clone session with a set of (`egress_port`, `instance`) pairs, similar to how a multicast group can be configured. Changes can be found here:

- Section 6.2. "Behavior of packets after ingress processing is complete"
- Section 6.4.5 "Multicast and clone copies" (formerly called "Multicast copies")
- Section 6.5 "Behavior of packets after egress processing is complete"
- Section 6.8 "Packet Cloning"

I.2.3. Add `psa_idle_timeout` table property

Adding this brings PSA v1.1 up to date with the support for this feature in the P4Runtime API. Using this table property enables the P4 developer to specify that a table must maintain some state of when the last time each table entry was matched, and if an entry remains unmatched for longer than a time configured by the controller, then a notification message should be sent to the controller.

- Section 7.2.1 "Table entry timeout notification"

I.2.4. Add `psa_empty_group_action` table property

PSA v1.0 did not specify the behavior of a table with an `ActionSelector` implementation, if a packet matched a table entry that was configured with an empty action selector group.

PSA v1.1 recommends (but does not require) that implementations support a new `psa_empty_group_action` table property, whose value is an action that should be executed when this situation occurs.

- Section 7.12 "Action Selector"

I.2.5. Other changes

In PSA v1.0, the `Digest` extern was required to be supported in both the `IngressDeparser` and `EgressDeparser` control blocks. It is now no longer required to be supported in the `EgressDeparser` control block.

- Table 5 "Summary of controls that can instantiate and invoke externs"

I.2.6. Changes to the `psa.p4` include file

- Updates for the latest plan on P4Runtime API numerical translation of type `PortId_t` and `ClassOfService_t`.
- Eliminate obsolete `ValueSet` extern, because the `value_set` construct was added to the P4₁₆ language specification in version 1.1.0.
- Fix a few typos in example control plane APIs in comments.
- Eliminate `PSA_SWITCH #define` macro with arguments, since the P4₁₆ language spec does not require that the P4₁₆ pre-processor support such macros.

I.2.7. Changes to example PSA programs in the `p4-16/psa/examples` directory

- Small changes required to bring them in harmony with the latest details on P4Runtime API numerical translation of type `PortId_t`.